

DDIM - generative process

Define a trainable generative process

$$p_{\theta}(x_{0:T}) := p_{\theta}(x_T) \prod_{i=1}^T p_{\theta}^{(t)}(x_{t-1}|x_t), \text{ where each } p_{\theta}^{(t)}(x_{t-1}|x_t) \text{ leverages knowledge of } q_{\sigma}(x_{t-1}|x_t, x_0).$$

1. Given a noisy observation X_t , e.g., $X_t = \sqrt{\alpha_t}X_0 + \sqrt{1 - \alpha_t}\epsilon_t$ with $X_0 \sim q(x_0)$ and $\epsilon_t \sim \mathcal{N}(0, I)$.

1) one step forward to add noise -> X_t .
(tricky*)

2. Make a prediction of the corresponding X_0 : The model $\epsilon_{\theta}^{(t)}(x_t)$ predicts ϵ_t from X_t , without knowing X_0 . Then we can predict the denoised observation, which is a prediction of X_0 given X_t ,

2) use diffusion model to predict the denoised observation X_0

$$f_{\theta}^{(t)}(x_t) := \frac{1}{\sqrt{\alpha_t}}(x_t - \sqrt{1 - \alpha_t}\epsilon_{\theta}^{(t)}(x_t)).$$

3. Use the prediction to obtain to sample X_{t-1} from the reverse conditional distribution

$q_{\sigma}(x_{t-1}|x_t, x_0)$: we can define the generative process with a fixed prior

$p_{\theta}(x_T) = \mathcal{N}(0, I)$ and

$$p_{\theta}^{(t)}(x_{t-1}|x_t) = \begin{cases} \mathcal{N}(f_{\theta}^{(1)}(x_1), \sigma_1^2 I) & \text{if } t = 1, \\ q_{\sigma}(x_{t-1}|x_t, f_{\theta}^{(t)}(x_t)) & \text{if } t \geq 1, \end{cases}$$

3) so we have X_t from step 1, and X_0 from step 2.
(tricky***)

$$q_{\sigma}(x_{t-1}|x_t, x_0) = \mathcal{N}\left(\sqrt{\alpha_{t-1}}x_0 + \sqrt{1 - \alpha_{t-1} - \sigma_t^2} \cdot \frac{x_t - \sqrt{\alpha_t}x_0}{\sqrt{1 - \alpha_t}}, \sigma_t^2 I\right) \quad x_0 = \frac{x_1 - \sqrt{1 - \alpha_1} \cdot \epsilon_0}{\sqrt{\alpha_1}} \quad q_{\sigma}(x_0 | x_1, f_{\theta}^{(1)}(x_1))$$

We can sample x_t at arbitrary time step t
in a closed form using reparameterization trick.

This is DDPM as we are using ϵ_t

$$q(x_t|x_{t-1}) := \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t \mathbf{I})$$



$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

A nice property of the above process is that we can sample $\mathbf{x}_t$ at any arbitrary time step t in a closed form using reparameterization trick. Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$:

$$\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{1 - \alpha_t}\epsilon_{t-1}$$

;where $\epsilon_{t-1}, \epsilon_{t-2}, \dots \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

(t-1-> t)

$$= \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1 - \alpha_t\alpha_{t-1}}\bar{\epsilon}_{t-2}$$

;where $\bar{\epsilon}_{t-2}$ merges two Gaussians (*).

(t-2 -> t-1 -> t) = ...

$$= \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

where,

$$\mathbf{x}_{t-1} = \sqrt{\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-2}$$

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I})$$

where,

$$\mathbf{x}_t = \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{\alpha_t(1 - \alpha_{t-1})}\epsilon_{t-2} + \sqrt{1 - \alpha_t}\epsilon_{t-1}$$

(*) Recall that when we merge two Gaussians with different variance, $\mathcal{N}(\mathbf{0}, \sigma_1^2\mathbf{I})$ and $\mathcal{N}(\mathbf{0}, \sigma_2^2\mathbf{I})$, the new distribution is $\mathcal{N}(\mathbf{0}, (\sigma_1^2 + \sigma_2^2)\mathbf{I})$. Here the merged standard deviation is

$$\sqrt{(1 - \alpha_t) + \alpha_t(1 - \alpha_{t-1})} = \sqrt{1 - \alpha_t\alpha_{t-1}}.$$

DDIM - generative process

Define a trainable generative process

$$p_\theta(x_{0:T}) := p_\theta(x_T) \prod_{i=1}^T p_\theta^{(t)}(x_{t-1}|x_t), \text{ where each } p_\theta^{(t)}(x_{t-1}|x_t) \text{ leverages knowledge of } q_\sigma(x_{t-1}|x_t, x_0).$$

Non-markovian forward: $q_\sigma(x_{1:T}|x_0) := q_\sigma(x_T|x_0) \prod_{t=2}^T q_\sigma(x_{t-1}|x_t, x_0)$

1. Given a noisy observation X_t , e.g., $X_t = \sqrt{\alpha_t}X_0 + \sqrt{1 - \alpha_t}\epsilon_t$ with $X_0 \sim q(x_0)$ and $\epsilon_t \sim \mathcal{N}(0, I)$.

one-step forward: $x_0 \rightarrow x_t$

2. Make a prediction of the corresponding X_0 : The model $\epsilon_\theta^{(t)}(x_t)$ predicts ϵ_t from X_t , without knowing X_0 . Then we can predict the denoised observation, which is a prediction of X_0 given X_t ,

given $x_t \rightarrow P(x_0|x_t)$ using diffusion model

$$f_\theta^{(t)}(x_t) := \frac{1}{\sqrt{\alpha_t}}(x_t - \sqrt{1 - \alpha_t}\epsilon_\theta^{(t)}(x_t)).$$

$$\begin{aligned} x_t &= \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_{t-1} & x_{t-1} &= \frac{x_t - \sqrt{1 - \alpha_t} \cdot \epsilon_{t-1}}{\sqrt{\alpha_t}} \\ x_1 &= \sqrt{\alpha_1} \cdot x_0 + \sqrt{1 - \alpha_1} \cdot \epsilon_0 & x_0 &= \frac{x_1 - \sqrt{1 - \alpha_1} \cdot \epsilon_0}{\sqrt{\alpha_1}} \end{aligned}$$

3. Use the prediction to obtain to sample X_{t-1} from the reverse conditional distribution $q_\sigma(x_{t-1}|x_t, x_0)$: we can define the generative process with a fixed prior $p_\theta(x_T) = \mathcal{N}(0, I)$ and

given x_0 (1.), x_t (2.) $\rightarrow x_{\{t-1\}}$ (3.)

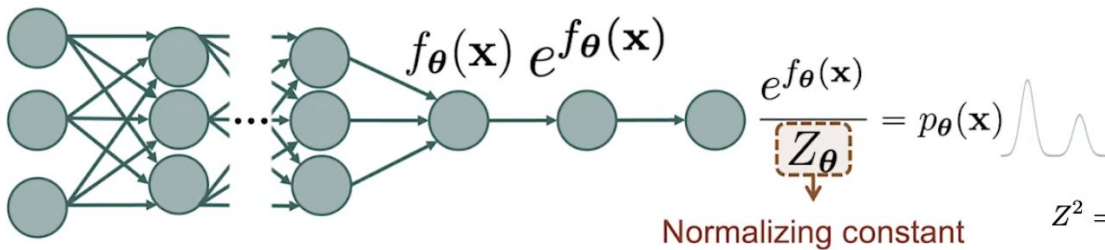
$$p_\theta^{(t)}(x_{t-1}|x_t) = \begin{cases} \mathcal{N}(f_\theta^{(1)}(x_1), \sigma_1^2 I) & \text{if } t = 1, \\ q_\sigma(x_{t-1}|x_t, f_\theta^{(t)}(x_t)) & \text{if } t \geq 1, \end{cases}$$

$$q_\sigma(x_0 | x_1, f_\theta^{(1)}(x_1))$$

Question 1 -



$\mathbf{x}$



The normalization constant for a zero-mean Gaussian

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(\frac{x-\mu}{\sigma})^2} \longrightarrow \int_{-\infty}^{\infty} f(x) dx = 1$$
$$Z = \int_a^b \exp\left(-\frac{x^2}{2\sigma^2}\right) dx$$
$$Z^2 = \int_a^b \int_a^b \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) dx dy$$
$$Z^2 = \int_0^{2\pi} \int_0^{\infty} r \exp\left(-\frac{r^2}{2\sigma^2}\right) dr d\theta \quad Z = \sigma\sqrt{(2\pi)}$$

Normalizing constant for Gaussians

$$Z_{\mu} = \frac{1}{(2\pi)^{d/2}}$$

Has close form - OK

$$Z_{\theta} = \int e^{f_{\theta}(\mathbf{x})} d\mathbf{x}$$

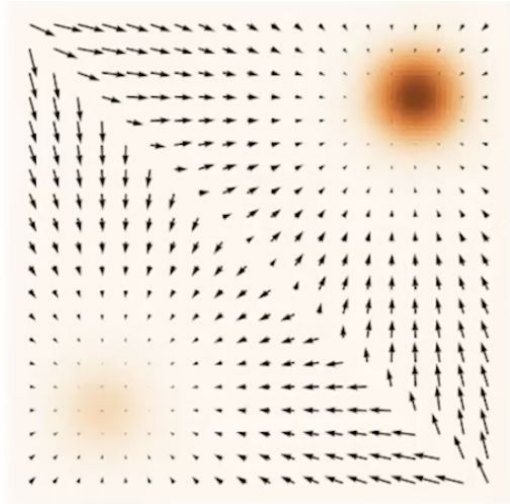
With more powerful NN, this is hard to compute - Nah

$p(\mathbf{x})$
Probability density function

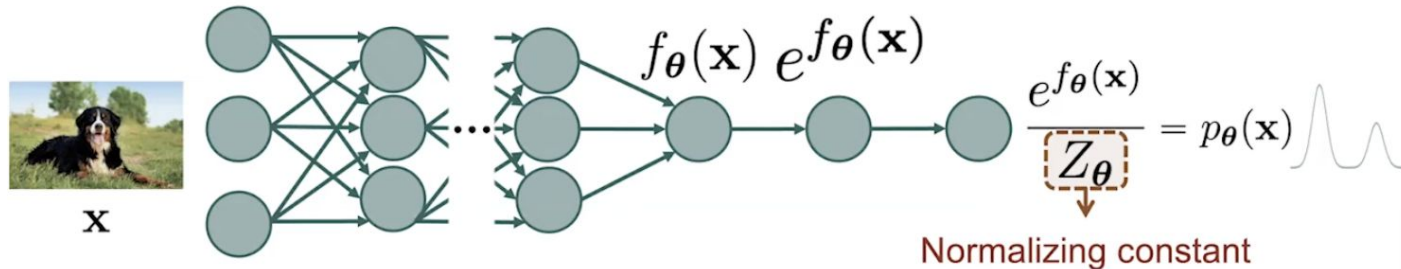
-> density map (the darker the high density)

-> vector field of direction where the density function grows most quickly

$\nabla_{\mathbf{x}} \log p(\mathbf{x})$
(Stein) score function



Score vs. density function



The idea of an energy-based model [Hinton, 1999] is, rather than explicitly learning a probabilistic model $p_{\theta}(x)$ over a space $\mathcal{X}$, to instead learn an energy functional $E_{\theta} : \mathcal{X} \rightarrow \mathbb{R}$. This energy functional can be used to implicitly define a probability distribution for example a Gibbs distribution

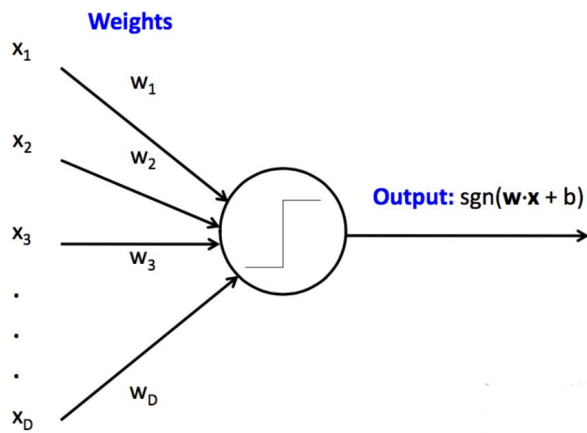
$$p_{\theta}(x) = \frac{1}{Z_{\theta}} e^{-E_{\theta}(x)}, \text{ where } Z_{\theta} = \int_{\mathcal{X}} e^{-E_{\theta}(y)} dy. \quad (1)$$

The point is that, while it is easy to construct a function $E_{\theta}(x)$, it can be quite challenging to enforce the constraint $\int_{\mathcal{X}} p_{\theta}(x) = 1$, or to compute the partition function Z_{θ} for a given energy function $E_{\theta}(x)$.

To use an energy-based model as a generative model, we need to solve two problems. First, we need a training procedure for optimizing the parameters of the energy function E_{θ} so that the implicit distribution $p_{\theta}(x)$ approximates the data generating distribution $p(x)$. And second, we need a sampling procedure for drawing samples $x \sim p_{\theta}$. Solutions to both problems should avoid calculation of the intractable integral Z_{θ} . For early approaches to this problem based on the contrastive divergence, see Hinton [2002] and Hinton et al. [2006]. For a modern, empirical realization of these ideas see Du and Mordatch [2019].

Perceptron

A **multilayer perceptron** is a special case of a feedforward neural network where every layer is a fully connected layer, the raw outputs from the final layer of a deep learning network are called logits.



The output of $f(x)$ is logit/coefficient/odd ratio, not probability. -> Logits can be negative.

Prediction: $f(x) = w_0x_0 + w_1x_1 + \dots w_mx_m + b$

$$\text{logit}(p) = \log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1x_1 + \dots + \beta_kx_k.$$

take exp() to make it positive

$$\frac{1-p}{p} = \frac{1}{\exp(\beta_0 + \beta_1x_1 + \dots + \beta_kx_k)}.$$

Do easy math you can then have probability

$$p = \frac{\exp(\beta_0 + \beta_1x_1 + \dots + \beta_kx_k)}{1 + \exp(\beta_0 + \beta_1x_1 + \dots + \beta_kx_k)}.$$

So it's sigmoid

	hon	Coef.
math		.1563404
intercept		-9.793942

$$\log(p/(1-p)) = \text{logit}(p) = -9.793942 + .1563404 \cdot \text{math}$$

$$\exp\left[\log\left(\frac{p}{1-p}\right)_{\text{math}=55} - \log\left(\frac{p}{1-p}\right)_{\text{math}=54}\right]$$

$$= \frac{\exp\left(\log\left(\frac{p}{1-p}\right)_{\text{math}=55}\right)}{\exp\left(\log\left(\frac{p}{1-p}\right)_{\text{math}=54}\right)}$$

$$= \frac{\text{odds}(\text{math} = 55)}{\text{odds}(\text{math} = 54)}$$

$$= \exp(0.1563404)$$

$$= 1.1692241$$

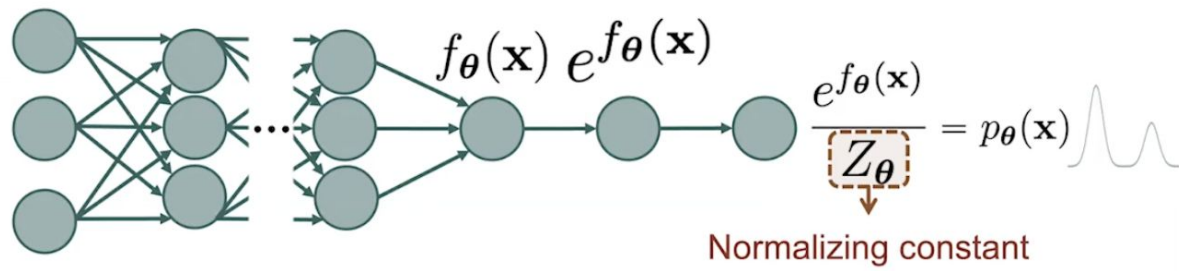
Perceptron = thresholded linear prediction model for classification

Very similar to binary logistic regression, though perceptron does not imply a particular error or training objective

For a one-unit increase in the math score, the expected change in log odds is .1563404.



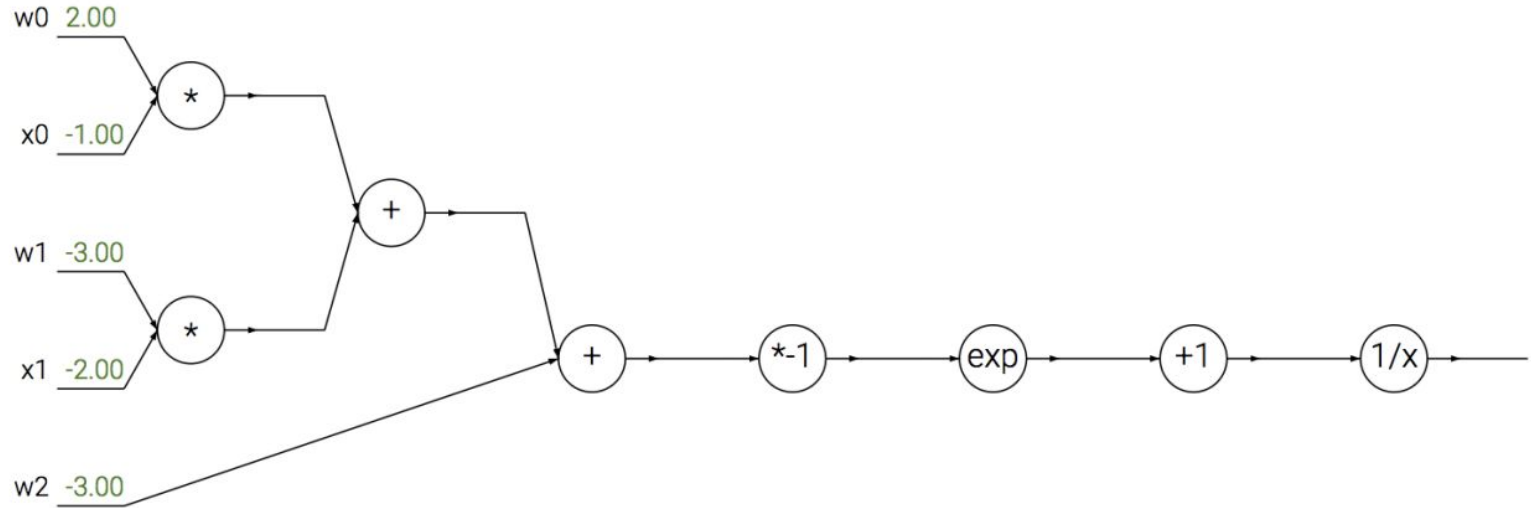
$\mathbf{x}$



For a one-unit increase in math score, we expect to see about 17% increase in the odds

A detailed example

$$f(x, w) = \frac{1}{1 + \exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



12.2 Logistic Regression

To sum up: we have a binary output variable Y , and we want to model the conditional probability $\Pr(Y = 1|X = x)$ as a function of x ; any unknown parameters in the function are to be estimated by maximum likelihood. By now, it will not surprise you to learn that statisticians have approached this problem by asking themselves “how can we use linear regression to solve this?”

1. The most obvious idea is to let $p(x)$ be a linear function of x . Every increment of a component of x would add or subtract so much to the probability. The conceptual problem here is that p must be between 0 and 1, and linear functions are unbounded. Moreover, in many situations we empirically see “diminishing returns” — changing p by the same amount requires a bigger change in x when p is already large (or small) than when p is close to $1/2$. Linear models can’t do this.
2. The next most obvious idea is to let $\log p(x)$ be a linear function of x , so that changing an input variable *multiplies* the probability by a fixed amount. The problem is that logarithms are unbounded in only one direction, and linear functions are not.
3. Finally, the easiest modification of $\log p$ which has an unbounded range is the logistic (or logit) transformation, $\log \frac{p}{1-p}$. We can make *this* a linear function of x without fear of nonsensical results. (Of course the results could still happen to be *wrong*, but they’re not *guaranteed* to be wrong.)

This last alternative is **logistic regression**.

Formally, the model logistic regression model is that

$$\log \frac{p(x)}{1-p(x)} = \beta_0 + x \cdot \beta \quad (12.4)$$

Solving for p , this gives

$$p(x; b, w) = \frac{e^{\beta_0 + x \cdot \beta}}{1 + e^{\beta_0 + x \cdot \beta}} = \frac{1}{1 + e^{-(\beta_0 + x \cdot \beta)}} \quad (12.5)$$

Notice that the over-all specification is a lot easier to grasp in terms of the transformed probability than in terms of the untransformed probability.¹

• Energies are uncalibrated

- ▶ The energies of two separately-trained systems cannot be combined
- ▶ The energies are uncalibrated (measured in arbitrary units)

• How do we calibrate energies?

- ▶ We turn them into probabilities (positive numbers that sum to 1).
- ▶ Simplest way: Gibbs distribution
- ▶ Other ways can be reduced to Gibbs by a suitable redefinition of the energy.

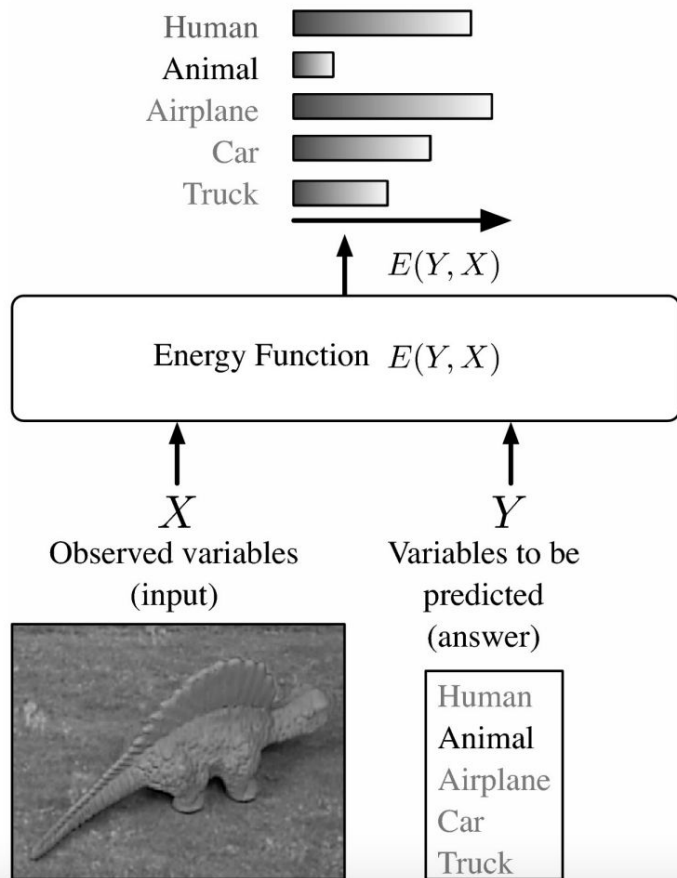
$$P(Y|X) = \frac{e^{-\beta E(Y,X)}}{\int_{y \in \mathcal{Y}} e^{-\beta E(y,X)}},$$

Partition function

Inverse temperature

¹Unless you’ve taken statistical mechanics, in which case you recognize that this is the Boltzmann distribution for a system with two states, which differ in energy by $\beta_0 + x \cdot \beta$.

Energy-Based Model for Decision-Making



Model: Measures the compatibility between an observed variable X and a variable to be predicted Y through an energy function $E(Y, X)$.

$$Y^* = \operatorname{argmin}_{Y \in \mathcal{Y}} E(Y, X).$$

Inference: Search for the Y that minimizes the energy within a set $\mathcal{Y}$

If the set has low cardinality, we can use exhaustive search.

Decision-Making versus Probabilistic Modeling

• Energies are uncalibrated

- ▶ The energies of two separately-trained systems cannot be combined
- ▶ The energies are uncalibrated (measured in arbitrary units)

• How do we calibrate energies?

- ▶ We turn them into probabilities (positive numbers that sum to 1).
- ▶ Simplest way: Gibbs distribution
- ▶ Other ways can be reduced to Gibbs by a suitable redefinition of the energy.

$$P(Y|X) = \frac{e^{-\beta E(Y,X)}}{\int_{y \in \mathcal{Y}} e^{-\beta E(y,X)}},$$

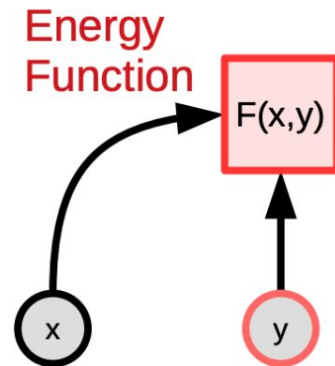
Partition function

Inverse temperature

Energy-Based Models vs Probabilistic Models

- ▶ **Probabilistic models are a special case of EBM**
 - ▶ Energies are like un-normalized negative log probabilities
- ▶ **Why use EBM instead of probabilistic models?**
 - ▶ EBM gives **more flexibility** in the choice of the scoring function.
 - ▶ **More flexibility** in the choice of objective function for learning
- ▶ **From energy to probability: Gibbs-Boltzmann distribution**
 - ▶ Beta is a positive constant

$$P(y|x) = \frac{e^{-\beta F(x,y)}}{\int_{y'} e^{-\beta F(x,y')}}$$



Contrastive Methods vs Regularized/Architectural Methods

- ▶ Contrastive: [different ways to pick which points to push up]
 - ▶ C1: push down of the energy of data points, push up everywhere else: **Max likelihood** (needs tractable partition function or variational approximation)
 - ▶ C2: push down of the energy of data points, push up on chosen locations: max likelihood with MC/MMC/HMC, Contrastive divergence, **Metric learning/Siamese nets**, Ratio Matching, Noise Contrastive Estimation, Min Probability Flow, **adversarial generator/GANs**
 - ▶ C3: train a function that maps points off the data manifold to points on the data manifold: denoising auto-encoder, **masked auto-encoder** (e.g. BERT)
- ▶ Regularized/Architectural: [Different ways to limit the information capacity of the latent representation]
 - ▶ A1: build the machine so that the volume of low energy space is upper-bounded: PCA, K-means, Gaussian Mixture Model, Square ICA, normalizing flows...
 - ▶ A2: use a regularization term that minimizes the volume of space that has low energy: Sparse coding, **sparse auto-encoder**, LISTA, Discretized AE/VQVAE, Contracting AE, Saturating AE, Noisy AE, **Variational AE**, SwAV, BYOL, Barlow Twins.
 - ▶ A4: minimize the gradient and maximize the curvature around data points: score matching

Score-Based Generative Modeling through Stochastic Differential Equations

Authors: Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, Ben Poole

Publication date: 2021/5/7 (ICLR)

Pages: 36

Main Pages: 5 (theory) + 4 (experiment)

TLDR:

- SDE and a corresponding reverse-time SDE;
- the reverse-time SDE depends only on the time-dependent gradient field (a.k.a., score) of the perturbed data distribution;
- an equivalent neural ODE that samples from the same distribution as the SDE, but additionally enables exact likelihood computation, and improved sampling efficiency.

This is an well-established topic - I will borrow slides from Youtube to introduce the idea first, then refer back to the original paper.

Background [3 - 24]

SMLD (DENOISING SCORE MATCHING WITH LANGEVIN DYNAMICS)

Score function, score matching, sliced score matching, denoising score matching, langevin dynamics sampling.

DDPM (DENOISING DIFFUSION PROBABILISTIC MODELS)

just quick review

The Paper (Something New):

SCORE-BASED GENERATIVE MODELING WITH SDES [19 - 42]

Make finite discrete process continuous

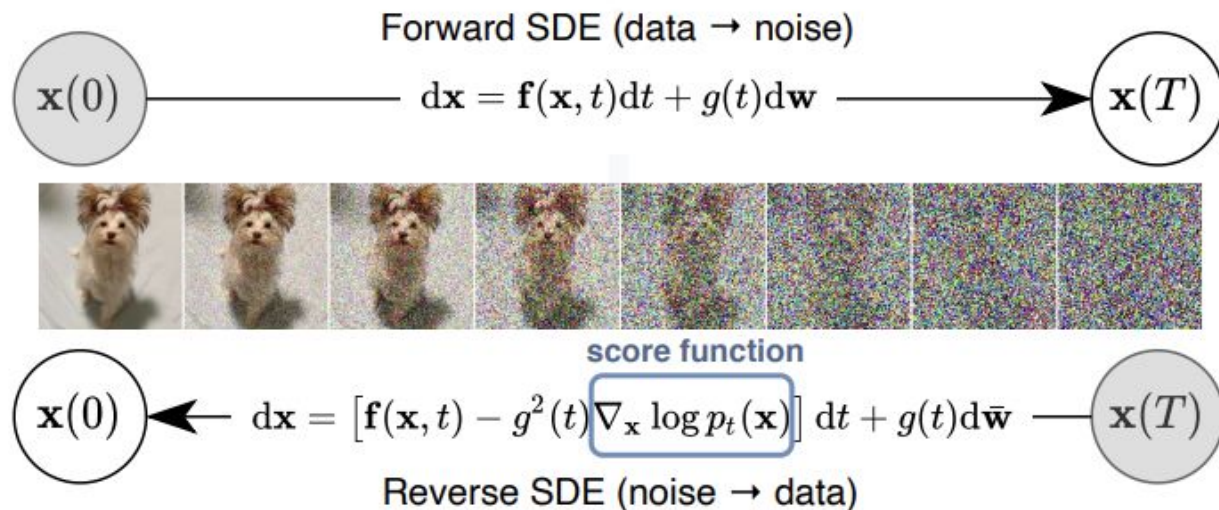


Figure 1: **Solving a reverse-time SDE yields a score-based generative model.** Transforming data to a simple noise distribution can be accomplished with a continuous-time SDE. This SDE can be reversed if we know the score of the distribution at each intermediate time step, $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$.

random noise into data for sample generation. Crucially, this reverse process satisfies a reverse-time SDE (Anderson, 1982), which can be derived from the forward SDE given the score of the marginal probability densities as a function of time. We can therefore approximate the reverse-time SDE by training a time-dependent neural network to estimate the scores, and then produce samples using numerical SDE solvers. Our key idea is summarized in Fig. 1.

Contributions

Score-based generative modeling: outline

Flexible models

- Bypass the normalizing constant
- Principled statistical methods

[Song et al. UAI 2019 *oral*]

Improved generation

- Higher sample quality than GANs
- Controllable generation

[Song & Ermon. NeurIPS 2019 *oral*]
[Song & Ermon. NeurIPS 2020]
[Song et al. ICLR 2021 *oral*]
(Outstanding Paper Award)
[Song et al. ICLR 2022]

Probability evaluation

- Accurate probability evaluation
- Better estimation of data probabilities

[Song et al. ICLR 2021 *oral*]
[Song et al. NeurIPS 2021 *spotlight*]

Contributions

Flexible sampling and likelihood computation: We can employ any general-purpose SDE solver to integrate the reverse-time SDE for sampling. In addition, we propose two special methods not viable for general SDEs: (i) Predictor-Corrector (PC) samplers that combine numerical SDE solvers with score-based MCMC approaches, such as Langevin MCMC (Parisi, 1981) and HMC (Neal et al., 2011); and (ii) deterministic samplers based on the probability flow ordinary differential equation (ODE). The former *unifies and improves* over existing sampling methods for score-based models.

Controllable generation: We can modulate the generation process by conditioning on information not available during training, because the conditional reverse-time SDE can be efficiently estimated from *unconditional* scores. This enables applications such as class-conditional generation, image inpainting, colorization and other inverse problems, all achievable using a single unconditional score-based model without re-training.

Unified framework: Our framework provides a unified way to explore and tune various SDEs for improving score-based generative models. The methods of SMLD and DDPM can be amalgamated into our framework as discretizations of two separate SDEs. Although DDPM (Ho et al., 2020) was recently reported to achieve higher sample quality than SMLD (Song & Ermon, 2019; 2020), we show that with better architectures and new sampling algorithms allowed by our framework, the latter can catch up—it achieves new state-of-the-art Inception score (9.89) and FID score (2.20) on CIFAR-10, as well as high-fidelity generation of 1024×1024 images for the first time from a score-based model. In addition, we propose a new SDE under our framework that achieves a likelihood value of 2.99 bits/dim on uniformly dequantized CIFAR-10 images, setting a new record on this task.

2.1 DENOISING SCORE MATCHING WITH LANGEVIN DYNAMICS (SMLD)

Let $p_\sigma(\tilde{\mathbf{x}} \mid \mathbf{x}) := \mathcal{N}(\tilde{\mathbf{x}}; \mathbf{x}, \sigma^2 \mathbf{I})$ be a perturbation kernel, and $p_\sigma(\tilde{\mathbf{x}}) := \int p_{\text{data}}(\mathbf{x}) p_\sigma(\tilde{\mathbf{x}} \mid \mathbf{x}) d\mathbf{x}$, where $p_{\text{data}}(\mathbf{x})$ denotes the data distribution. Consider a sequence of positive noise scales $\sigma_{\min} = \sigma_1 < \sigma_2 < \dots < \sigma_N = \sigma_{\max}$. Typically, $\sigma_{\min}$ is small enough such that $p_{\sigma_{\min}}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$, and $\sigma_{\max}$ is large enough such that $p_{\sigma_{\max}}(\mathbf{x}) \approx \mathcal{N}(\mathbf{x}; \mathbf{0}, \sigma_{\max}^2 \mathbf{I})$. Song & Ermon (2019) propose to train a Noise Conditional Score Network (NCSN), denoted by $\mathbf{s}_\theta(\mathbf{x}, \sigma)$, with a weighted sum of denoising score matching (Vincent, 2011) objectives:

$$\mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2 \right] \quad \text{DDPM} \quad (12)$$

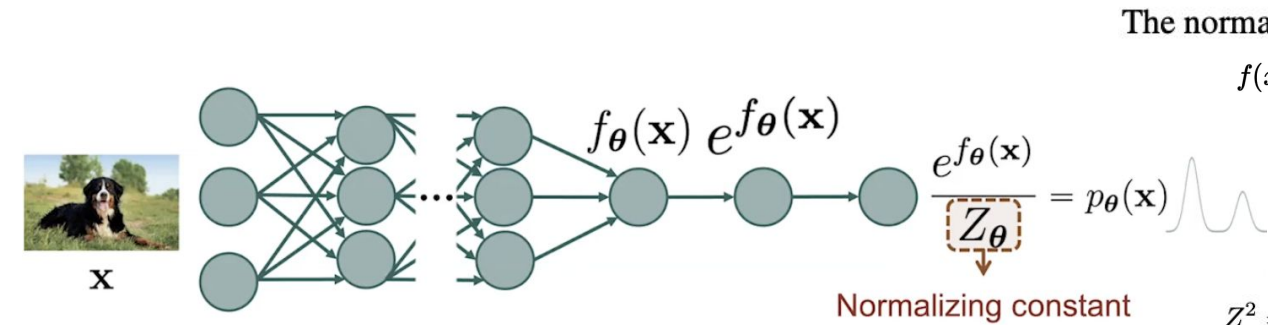
$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N \sigma_i^2 \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{p_{\sigma_i}(\tilde{\mathbf{x}} \mid \mathbf{x})} [\|\mathbf{s}_\theta(\tilde{\mathbf{x}}, \sigma_i) - \nabla_{\tilde{\mathbf{x}}} \log p_{\sigma_i}(\tilde{\mathbf{x}} \mid \mathbf{x})\|_2^2]. \quad (1)$$

Given sufficient data and model capacity, the optimal score-based model $\mathbf{s}_{\theta^*}(\mathbf{x}, \sigma)$ matches $\nabla_{\mathbf{x}} \log p_\sigma(\mathbf{x})$ almost everywhere for $\sigma \in \{\sigma_i\}_{i=1}^N$. For sampling, Song & Ermon (2019) run M steps of Langevin MCMC to get a sample for each $p_{\sigma_i}(\mathbf{x})$ sequentially:

$$\mathbf{x}_i^m = \mathbf{x}_i^{m-1} + \epsilon_i \mathbf{s}_{\theta^*}(\mathbf{x}_i^{m-1}, \sigma_i) + \sqrt{2\epsilon_i} \mathbf{z}_i^m, \quad m = 1, 2, \dots, M, \quad (2)$$

where $\epsilon_i > 0$ is the step size, and $\mathbf{z}_i^m$ is standard normal. The above is repeated for $i = N, N - 1, \dots, 1$ in turn with $\mathbf{x}_N^0 \sim \mathcal{N}(\mathbf{x} \mid \mathbf{0}, \sigma_{\max}^2 \mathbf{I})$ and $\mathbf{x}_i^0 = \mathbf{x}_{i+1}^M$ when $i < N$. As $M \rightarrow \infty$ and $\epsilon_i \rightarrow 0$ for all i , $\mathbf{x}_1^M$ becomes an exact sample from $p_{\sigma_{\min}}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$ under some regularity conditions.

SMLD - intractable normalizing constant & (stein) score function 1/14



The normalization constant for a zero-mean Gaussian

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \longrightarrow \int_{-\infty}^{\infty} f(x) dx = 1$$
$$Z = \int_a^b \exp\left(-\frac{x^2}{2\sigma^2}\right) dx$$
$$Z^2 = \int_a^b \int_a^b \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right) dx dy$$
$$Z^2 = \int_0^{2\pi} \int_0^{\infty} r \exp\left(-\frac{r^2}{2\sigma^2}\right) dr d\theta \quad Z = \sigma\sqrt{(2\pi)}$$

Normalizing constant for Gaussians

$$Z_{\mu} = \frac{1}{(2\pi)^{d/2}}$$

Has close form - OK

$$Z_{\theta} = \int e^{f_{\theta}(\mathbf{x})} d\mathbf{x}$$

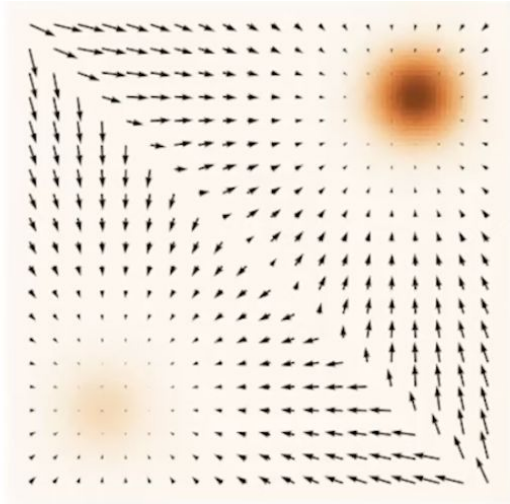
With more powerful NN, this is hard to compute - Nah

$p(\mathbf{x})$
Probability density function

-> density map (the darker the high density)

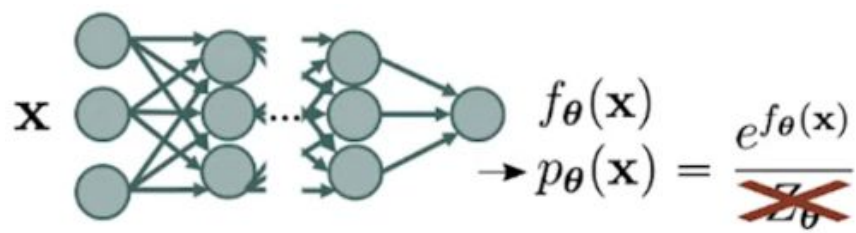
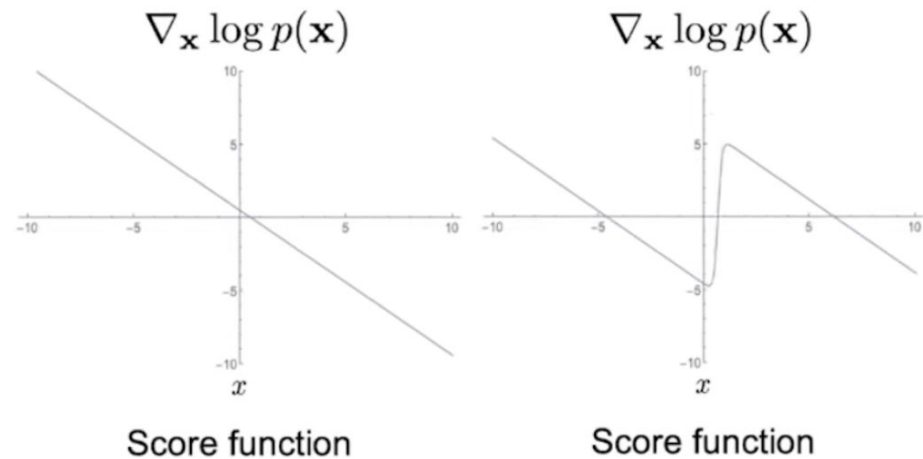
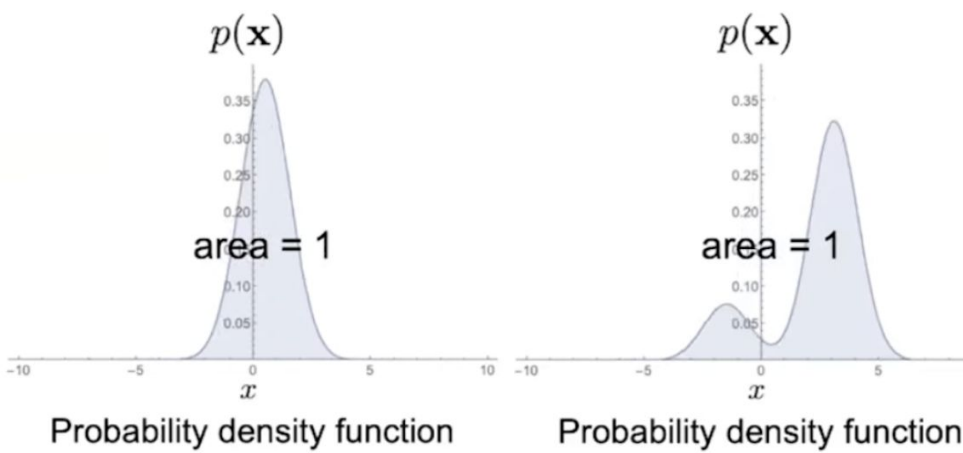
-> vector field of direction where the density function grows most quickly

$\nabla_{\mathbf{x}} \log p(\mathbf{x})$
(Stein) score function



Score vs. density function

SMLD - intractable normalizing constant & (stein) score function 2/14



Intractable Z

$$\begin{aligned}\nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}) &= \nabla_{\mathbf{x}} f_{\theta}(\mathbf{x}) - \nabla_{\mathbf{x}} \log Z_{\theta} \\ &= \nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})\end{aligned}$$

0

No such problem and informative enough

SMLD - Score Model 3/14

The key idea is to model the **gradient of the log probability density function**, a quantity often known as the (Stein) **score function**.

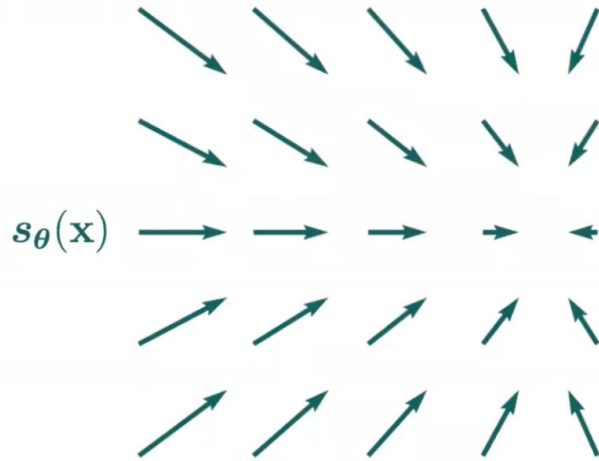
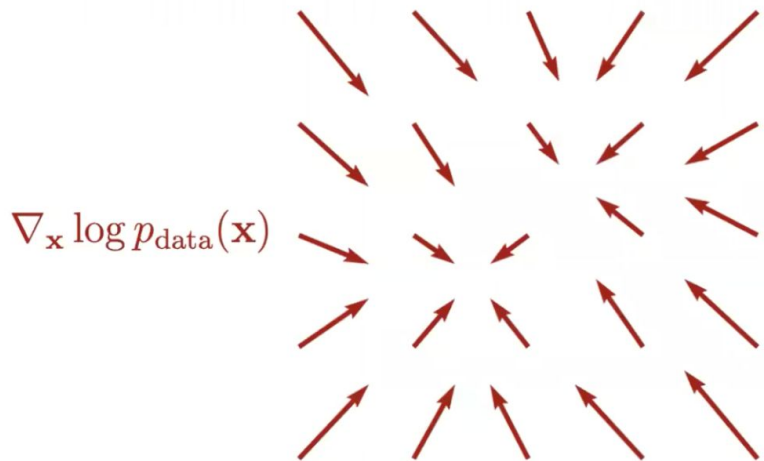
Such score-based models are not required to have a tractable **normalizing constant**, and can be directly learned by score matching

Given: $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\} \stackrel{\text{i.i.d.}}{\sim} p_{\text{data}}(\mathbf{x})$

Goal: $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Score Model: $s_{\theta}(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^d \approx \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$

Objective: How to compare two vector fields of scores?



$$\hat{\theta} = \arg \max_{\theta} L(\theta|X)$$

$$L(\theta|X) = P(X|\theta) = \prod_{i=1}^n P(x_i|\theta)$$

$$\ell(\theta|X) = \log L(\theta|X) = \sum_{i=1}^n \log P(x_i|\theta)$$

$$\nabla_{\theta} \ell(\theta|X) = \left(\frac{\partial}{\partial \theta_1} \ell(\theta|X), \frac{\partial}{\partial \theta_2} \ell(\theta|X), \dots, \frac{\partial}{\partial \theta_p} \ell(\theta|X) \right)$$

$$\frac{\partial}{\partial \theta} \ell(\theta|X) = 0$$

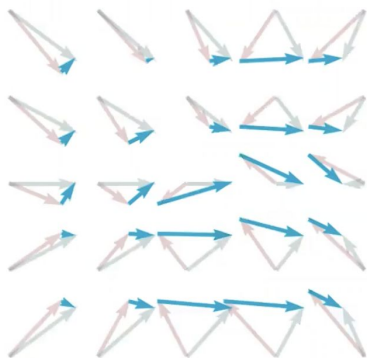
SMLD - Score Matching 4/14

$\nabla_{\mathbf{x}} \log P_{\text{data}}(\mathbf{x})$ is not tractable

$$\frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x})\|_2^2]$$

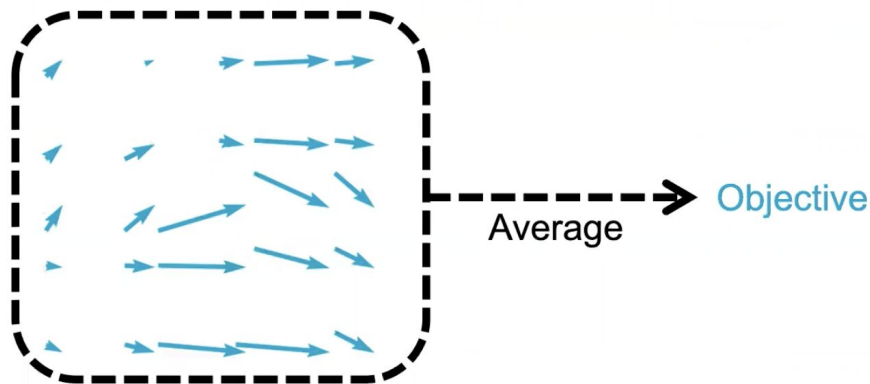
(Fisher divergence)

Idea: Compare the difference vectors between those pairs of vectors



Score Matching [Hyvärinen 2005]:

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \left[\frac{1}{2} \|\mathbf{s}_{\theta}(\mathbf{x})\|_2^2 + \text{trace} \left(\underbrace{\nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x})}_{\text{Jacobian of } \mathbf{s}_{\theta}(\mathbf{x})} \right) \right] \leftarrow$$
$$\approx \frac{1}{N} \sum_{i=1}^N \left[\frac{1}{2} \|\mathbf{s}_{\theta}(\mathbf{x}_i)\|_2^2 + \text{trace}(\nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x}_i)) \right]$$



the expected squared euclidean distance between the data score and the model score, averaged over samples from the data distribution.

SMLD - Score Matching 5/14

$$\frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} [\| \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}) \|_2^2]$$

(Fisher divergence)

$$\frac{1}{2} E_{P_{\text{data}}} \left[(\nabla_x \log P_{\text{data}}(x) - \nabla_x \log P_{\theta}(x))^2 \right]$$
$$= \frac{1}{2} \int P_{\text{data}}(x) (\nabla_x \log P_{\text{data}}(x))^2 dx = \text{constant (not about } \theta)$$

$$+ \frac{1}{2} \int P_{\text{data}}(x) (\nabla_x \log P_{\theta}(x))^2 dx$$
$$- \int P_{\text{data}}(x) \nabla_x \log P_{\theta}(x) \nabla_x \log P_{\text{data}}(x) dx$$
$$= - \int \nabla_x \log P_{\theta}(x) \cdot \nabla_x P_{\text{data}}(x) dx$$
$$= -P_{\text{data}}(x) \nabla_x \log P_{\theta}(x) \Big|_{-\infty}^{\infty} + \int P_{\text{data}}(x) \nabla_x^2 \log P_{\theta}(x) dx$$

Score Matching [Hyvärinen 2005]:

$$\mathbb{E}_{p_{\text{data}}(\mathbf{x})} \left[\frac{1}{2} \|\mathbf{s}_{\theta}(\mathbf{x})\|_2^2 + \text{trace} \left(\underbrace{\nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x})}_{\text{Jacobian of } \mathbf{s}_{\theta}(\mathbf{x})} \right) \right]$$
$$\approx \frac{1}{N} \sum_{i=1}^N \left[\frac{1}{2} \|\mathbf{s}_{\theta}(\mathbf{x}_i)\|_2^2 + \text{trace}(\nabla_{\mathbf{x}} \mathbf{s}_{\theta}(\mathbf{x}_i)) \right]$$

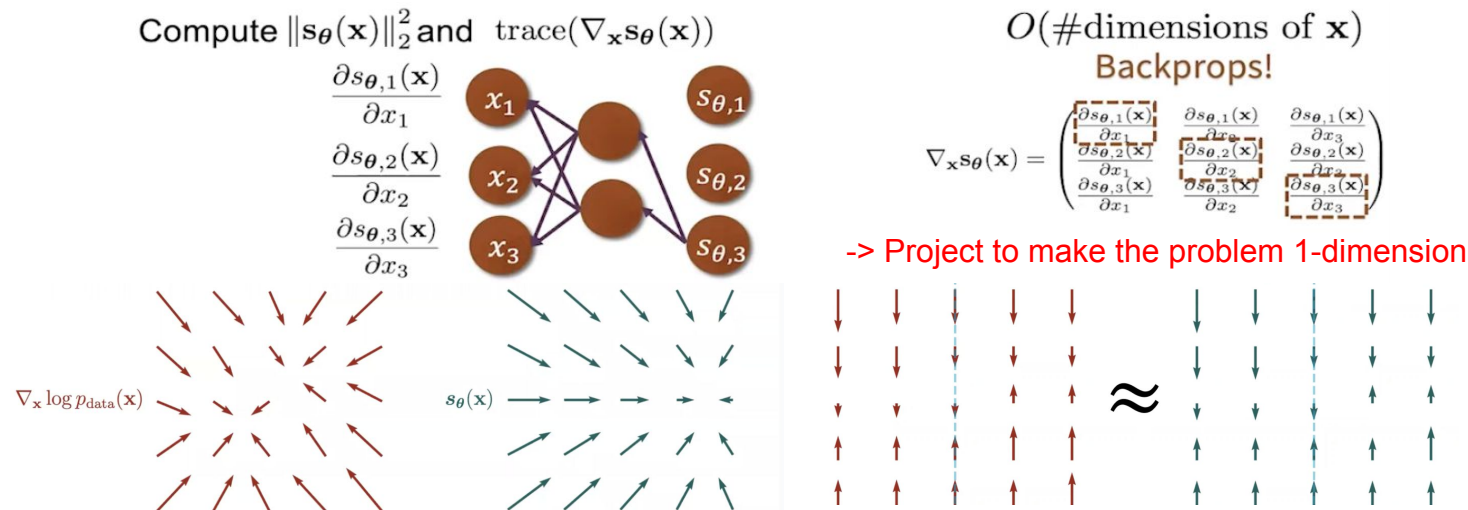
$$= E_{P_{\text{data}}} [\nabla_x^2 \log P_{\theta}(x)] \quad \text{(i) holds if } P_{\text{data}}(x) \rightarrow 0 \text{ when } |x| \rightarrow \infty.$$

$$\frac{1}{2} E_{P_{\text{data}}} [(\nabla_x \log P_{\text{data}}(x) - \nabla_x \log P_{\theta}(x))^2]$$
$$= E_{P_{\text{data}}} [\nabla_x^2 \log P_{\theta}(x)] + \frac{1}{2} E_{P_{\text{data}}} [(\nabla_x \log P_{\theta}(x))^2] + C$$

This equivalent from 1D Fisher Divergence
doesn't include $\nabla_x \log P_{\text{data}}(x)$!
No intractable term anymore

SMLD - Faster Score Matching 6/14

Score Matching is not scalable



Mathematically, we denote $\mathbf{v}$ as the random projection direction, and $p_{\mathbf{v}}$ as its distribution. The random projected version of Fisher divergence is

$$\frac{1}{2} \mathbb{E}_{p_{\text{data}}} [(\mathbf{v}^{\top} \nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x}) - \mathbf{v}^{\top} \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}))^2], \quad \text{This is scalable! no trace of jacobian}$$

which we name as the *sliced Fisher divergence*. Unfortunately, sliced Fisher divergence has the same problem as Fisher divergence, due to the unknown data score function $\nabla_{\mathbf{x}} \log p_{\text{data}}(\mathbf{x})$.

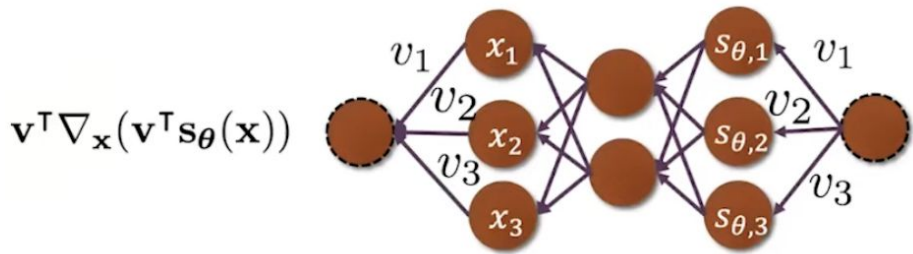
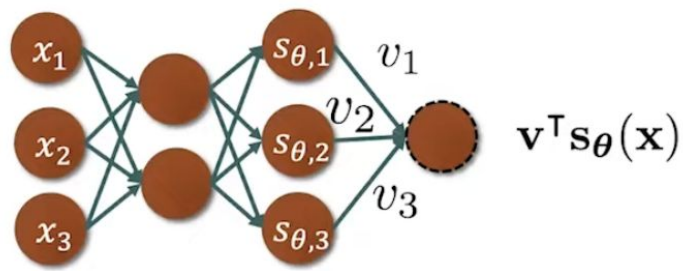
We therefore play the same trick of integration by parts, as done in score matching, to obtain the following tractable alternative form

$$\mathbb{E}_{p_{\text{data}}} \left[\mathbf{v}^{\top} \nabla_{\mathbf{x}}^2 \log p_{\theta}(\mathbf{x}) \mathbf{v} + \frac{1}{2} (\mathbf{v}^{\top} \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}))^2 \right] + \text{const}, \quad \text{N backprops -> 1 backprops}$$

SMLD - Faster Score Matching 7/14

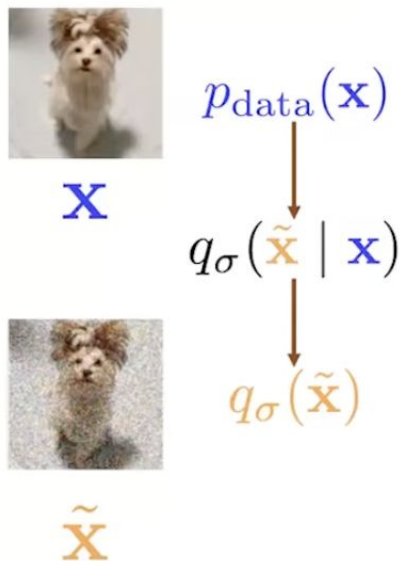
$$\mathbf{v}^\top \nabla_{\mathbf{x}} \mathbf{s}_\theta(\mathbf{x}) \mathbf{v} = \mathbf{v}^\top \nabla_{\mathbf{x}} (\mathbf{v}^\top \mathbf{s}_\theta(\mathbf{x}))$$

$$\mathbf{v}^\top \nabla_{\mathbf{x}} \mathbf{s}_\theta(\mathbf{x}) \mathbf{v} = \mathbf{v}^\top \nabla_{\mathbf{x}} (\mathbf{v}^\top \mathbf{s}_\theta(\mathbf{x}))$$



SMLD - Denoising score matching 8/14

- Add additional noise to the data point to help us to compute the choice of Jacobian term
- As the sigma is small, you can approximately view the score function of the noise density function == score function of the noise free density function



- Matching the score of a noise-perturbed distribution

$$\frac{1}{2} \mathbb{E}_{q_{\sigma}(\tilde{\mathbf{x}})} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}) - \mathbf{s}_{\theta}(\tilde{\mathbf{x}})\|_2^2]$$

- Denoising score matching (Vincent 2011)

$$\frac{1}{2} \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x})} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) - \mathbf{s}_{\theta}(\tilde{\mathbf{x}})\|_2^2]$$

↓
Scalable!

Perturbation kernel is design by hand, so it's easy to make it a tractable distribution - analytically ok

But you can't choose sigma too small - if you try to make a lower magnitude of the noise, the variance of denoising score matching objective will becomes bigger and bigger and eventually explode.

SMLD - Denoising score matching 9/14

- Sample a minibatch of datapoints $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \sim p_{\text{data}}(\mathbf{x})$
- Sample a minibatch of perturbed datapoints $\{\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \dots, \tilde{\mathbf{x}}_n\} \sim q_{\sigma}(\tilde{\mathbf{x}})$

$$\tilde{\mathbf{x}}_i \sim q_{\sigma}(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)$$

- Estimate the denoising score matching loss with empirical means

$$\frac{1}{2n} \sum_{i=1}^n [\|s_{\theta}(\tilde{\mathbf{x}}_i) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}_i \mid \mathbf{x}_i)\|_2^2]$$

- If Gaussian perturbation

$$\frac{1}{2n} \sum_{i=1}^n \left[\left\| s_{\theta}(\tilde{\mathbf{x}}_i) + \frac{\tilde{\mathbf{x}}_i - \mathbf{x}_i}{\sigma^2} \right\|_2^2 \right]$$

- Stochastic gradient descent
- Need to choose a very small σ !

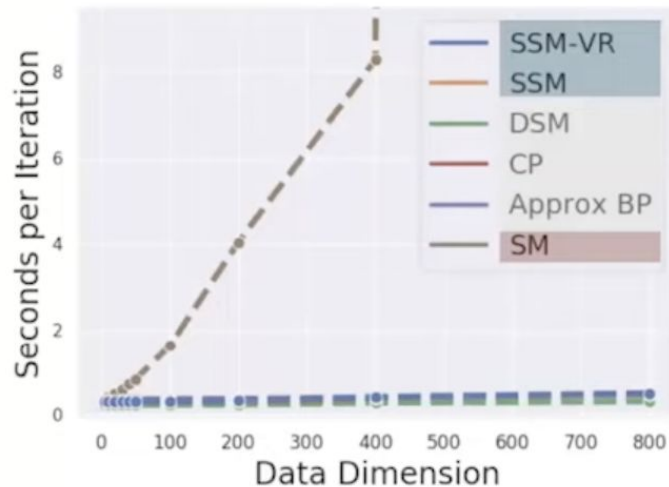
Results: Sliced Score Matching for EBM

Sliced score matching methods

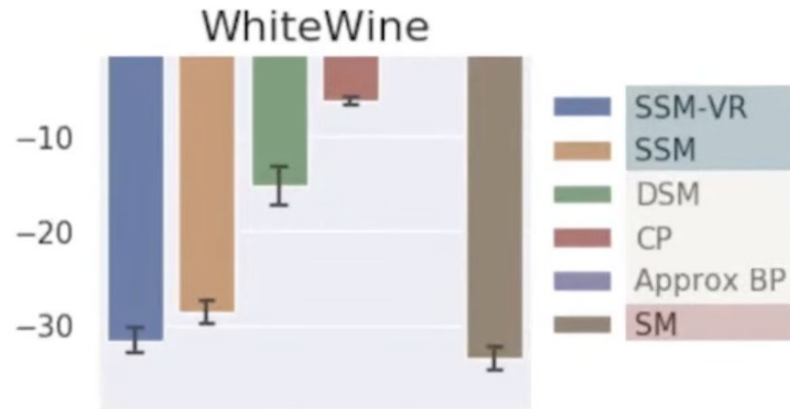
Other Baselines

Score Matching

how many times it takes in each iteration



Efficiency



Performance on density estimation

The lower the better

Score-based generative modeling: outline



Flexible models

- Bypass the normalizing constant
- Principled statistical methods

[Song et al. UAI 2019 oral]

Improved generation

- Higher sample quality than GANs
- Controllable generation

[Song & Ermon. NeurIPS 2019 oral]
[Song & Ermon. NeurIPS 2020]
[Song et al. ICLR 2021 oral]
(Outstanding Paper Award)
[Song et al. ICLR 2022]

Probability evaluation

- Accurate probability evaluation
- Better estimation of data probabilities

[Song et al. ICLR 2021 oral]
[Song et al. NeurIPS 2021 spotlight]

SMLD - Sampling, Langevin Dynamics 10/14

Once we have trained a score-based model $\mathbf{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$, we can use an iterative procedure called **Langevin dynamics** [32, 33] to draw samples from it.

Langevin dynamics provides an MCMC procedure to sample from a distribution $p(\mathbf{x})$ using only its score function $\nabla_{\mathbf{x}} \log p(\mathbf{x})$. Specifically, it initializes the chain from an arbitrary prior distribution $\mathbf{x}_0 \sim \pi(\mathbf{x})$, and then iterates the following

$$\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + \epsilon \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sqrt{2\epsilon} \mathbf{z}_i, \quad i = 0, 1, \dots, K, \quad (6)$$

where $\mathbf{z}_i \sim \mathcal{N}(\mathbf{0}, I)$. When $\epsilon \rightarrow 0$ and $K \rightarrow \infty$, $\mathbf{x}_K$ obtained from the procedure in (6) converges to a sample from $p(\mathbf{x})$ under some regularity conditions. In practice, the error is negligible when ϵ is sufficiently small and K is sufficiently large.

Note that Langevin dynamics accesses $p(\mathbf{x})$ only through $\nabla_{\mathbf{x}} \log p(\mathbf{x})$. Since $\mathbf{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$, we can produce samples from our score-based model $\mathbf{s}_\theta(\mathbf{x})$ by plugging it into equation (6).

SMLD - Langevin Dynamics math 11/14

Ito Stochastic Differential Equation (SDE):

$$dX(t) = -\nabla U(X(t))dt + \sigma dB_t$$

Discrete approximation:

$$X_{t+1} - X_t = -dt \nabla U(X_t) + \sigma \sqrt{dt} \mathcal{N}(0, 1)$$

$$W_{t+1} - W_t = -\varepsilon \nabla \mathcal{L}(W_t) + \sigma \sqrt{\varepsilon} \mathcal{N}(0, 1)$$

gradient descent

Derivation of the Fokker-Planck equation

Langevin equation:

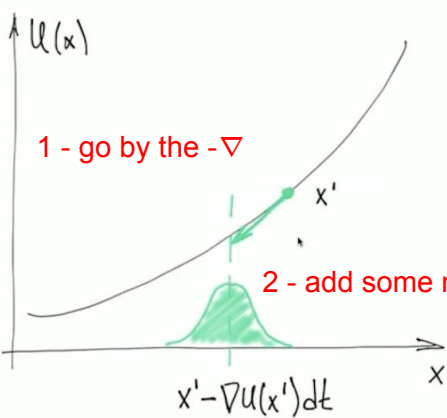
$$dX(t) = -\nabla U(X(t))dt + \sigma dB_t$$

Increments of the Brownian motion:

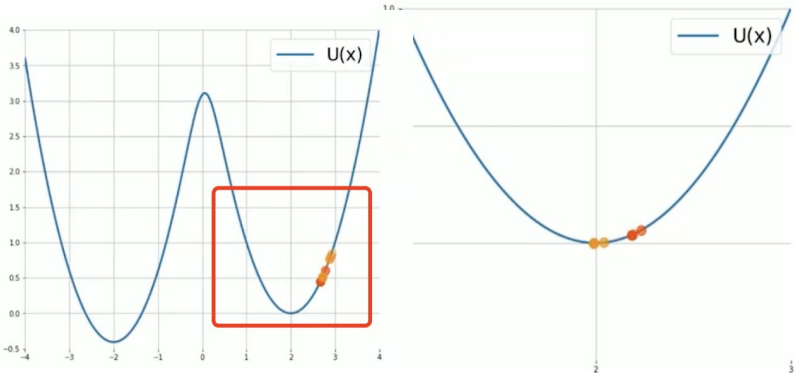
$$dB_t \sim \mathcal{N}(0, dt \cdot I)$$

Consider a small increment of $X(t)$:

$$x - x' = -dt \nabla U(x') + \mathcal{N}(0, \sigma^2 dt)$$

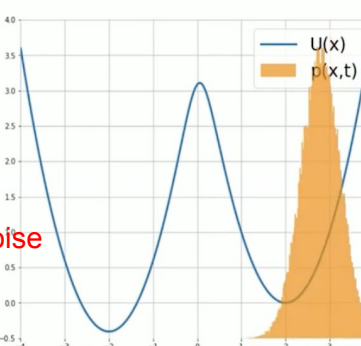


Langevin equation The movement of single particles

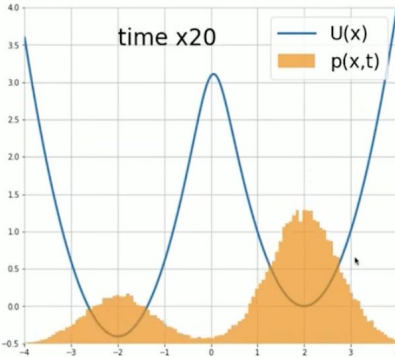


Particles are not stable (can't describe the behavior of these particles deterministically) due to stochasticity

Fokker-Planck equation



Alternative perspective R.V.



SMLD - Langevin Dynamics sampling math 12/14

Sampling via the Langevin dynamics

$$dX(t) = -\nabla U(X(t))dt + \sigma dB_t \quad \text{Langevin equation}$$

Particles have the stationary distribution:

$$p_G(x) = \frac{1}{Z} \exp\left(-\frac{2U(x)}{\sigma^2}\right), \quad Z = \int dx \exp\left(-\frac{2U(x)}{\sigma^2}\right) \quad \text{Gibbs distribution}$$

We want to sample from $p(x) = \frac{\hat{p}(x)}{Z'}$

$$U(x) = -\log p(x), \quad \sigma = \sqrt{2}$$

$$p_G(x) = \frac{1}{Z} \exp\left(-\frac{-2\log p(x)}{2}\right) = p(x), \quad Z = \int dx \exp(\log p(x)) = 1$$

Note! $\nabla U(x) = -\nabla \log p(x) = -\nabla \log \hat{p}(x) - \nabla \log Z'$

The gradient of $U(x)$ (energy), does not depend on Z' .

The Langevin function give us some complicated dynamics for us to simulate

Sampling via the Langevin dynamics

Stochastic Differential Equation for sampling:

$$dX(t) = \nabla \log p(X(t))dt + \sqrt{2}dB_t$$

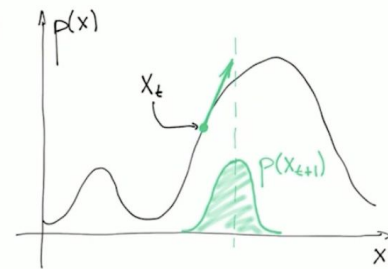
Discrete approximation:

$$X_{t+1} = X_t + dt \nabla \log p(X_t) + \mathcal{N}(0, 2dt)$$

More popular way:

$$X_{t+1} = X_t + \frac{\varepsilon}{2} \nabla \log p(X_t) + \mathcal{N}(0, \varepsilon)$$

$$X_t \sim p(x), \quad \forall t > t_\infty$$



The gradient of $U(x)$ (energy), does not depend on Z' .

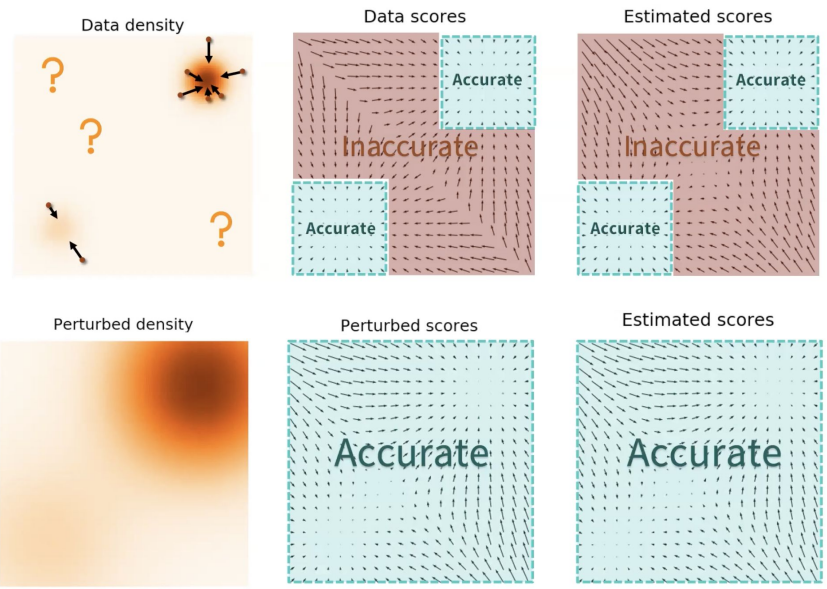
X moves to higher region of density

$$\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + \epsilon \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sqrt{2\epsilon} \mathbf{z}_i, \quad i = 0, 1, \dots, K, \quad (6)$$

SMLD - Sampling, challenge in low density region 13/14

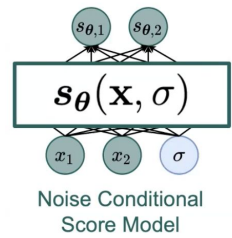
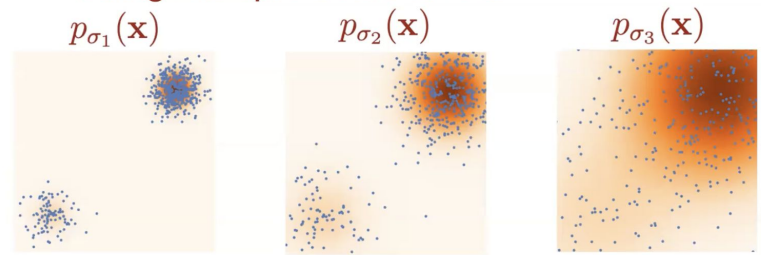
Perturb dataset (inject right amount of gaussian noise) -> populate low data density regions to shrink the size of low data density regions.

- A generalization to the training objective of diffusion probabilistic models [Sohl-Dickstein et al. 2015]
- First unveiled by DDPM [Ho et al. 2020]



Data

Using multiple noise levels



Positive weighting function

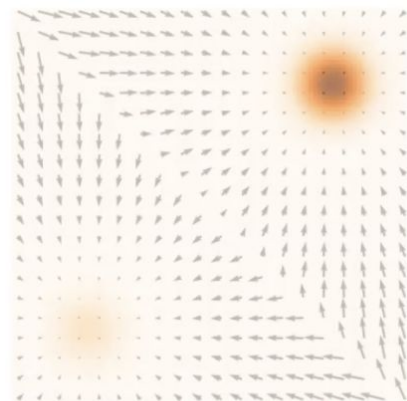
$$\frac{1}{N} \sum_{i=1}^N \lambda(\sigma_i) \mathbb{E}_{p_{\sigma_i}(\mathbf{x})} [\underbrace{\|\nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}, \sigma_i)\|_2^2}_{\text{Score matching loss}}]$$

We have one score matching loss for each noise level (sigma_i)

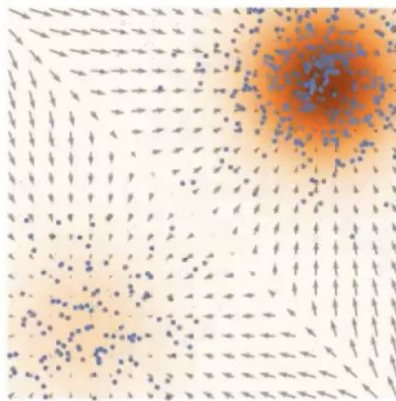
SMLD - Sampling - 14/14

$$\frac{1}{N} \sum_{i=1}^N \overbrace{\lambda(\sigma_i)}^{\text{Positive weighting function}} \underbrace{\mathbb{E}_{p_{\sigma_i}(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_{\sigma_i}(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}, \sigma_i)\|_2^2]}_{\text{Score matching loss}}$$

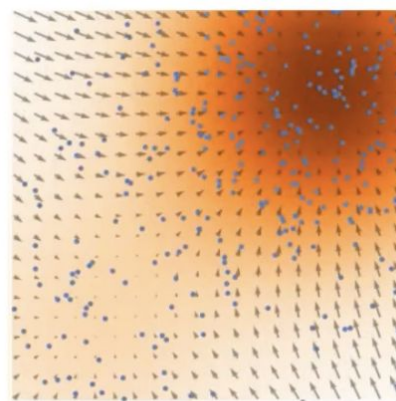
use langevin dynamics to sample from the score model with biggest noise level (n), then, use that samples as initialization to sample from the next noise level (n-1).



$\mathbf{s}_{\theta}(\mathbf{x}, \sigma_1)$



$\mathbf{s}_{\theta}(\mathbf{x}, \sigma_2)$



$\mathbf{s}_{\theta}(\mathbf{x}, \sigma_3)$

2.1 DENOISING SCORE MATCHING WITH LANGEVIN DYNAMICS (SMLD) - revisit

Let $p_\sigma(\tilde{\mathbf{x}} \mid \mathbf{x}) := \mathcal{N}(\tilde{\mathbf{x}}; \mathbf{x}, \sigma^2 \mathbf{I})$ be a perturbation kernel, and $p_\sigma(\tilde{\mathbf{x}}) := \int p_{\text{data}}(\mathbf{x}) p_\sigma(\tilde{\mathbf{x}} \mid \mathbf{x}) d\mathbf{x}$, where $p_{\text{data}}(\mathbf{x})$ denotes the data distribution. Consider a sequence of positive noise scales $\sigma_{\min} = \sigma_1 < \sigma_2 < \dots < \sigma_N = \sigma_{\max}$. Typically, $\sigma_{\min}$ is small enough such that $p_{\sigma_{\min}}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$, and $\sigma_{\max}$ is large enough such that $p_{\sigma_{\max}}(\mathbf{x}) \approx \mathcal{N}(\mathbf{x}; \mathbf{0}, \sigma_{\max}^2 \mathbf{I})$. Song & Ermon (2019) propose to train a Noise Conditional Score Network (NCSN), denoted by $\mathbf{s}_\theta(\mathbf{x}, \sigma)$, with a weighted sum of denoising score matching (Vincent, 2011) objectives:

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N \sigma_i^2 \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{p_{\sigma_i}(\tilde{\mathbf{x}} \mid \mathbf{x})} \left[\|\mathbf{s}_\theta(\tilde{\mathbf{x}}, \sigma_i) - \nabla_{\tilde{\mathbf{x}}} \log p_{\sigma_i}(\tilde{\mathbf{x}} \mid \mathbf{x})\|_2^2 \right]. \quad (1)$$

Given sufficient data and model capacity, the optimal score-based model $\mathbf{s}_{\theta^*}(\mathbf{x}, \sigma)$ matches $\nabla_{\mathbf{x}} \log p_\sigma(\mathbf{x})$ almost everywhere for $\sigma \in \{\sigma_i\}_{i=1}^N$. For sampling, Song & Ermon (2019) run M steps of Langevin MCMC to get a sample for each $p_{\sigma_i}(\mathbf{x})$ sequentially:

$$\mathbf{x}_i^m = \mathbf{x}_i^{m-1} + \epsilon_i \mathbf{s}_{\theta^*}(\mathbf{x}_i^{m-1}, \sigma_i) + \sqrt{2\epsilon_i} \mathbf{z}_i^m, \quad m = 1, 2, \dots, M, \quad (2)$$

where $\epsilon_i > 0$ is the step size, and $\mathbf{z}_i^m$ is standard normal. The above is repeated for $i = N, N-1, \dots, 1$ in turn with $\mathbf{x}_N^0 \sim \mathcal{N}(\mathbf{x} \mid \mathbf{0}, \sigma_{\max}^2 \mathbf{I})$ and $\mathbf{x}_i^0 = \mathbf{x}_{i+1}^M$ when $i < N$. As $M \rightarrow \infty$ and $\epsilon_i \rightarrow 0$ for all i , $\mathbf{x}_1^M$ becomes an exact sample from $p_{\sigma_{\min}}(\mathbf{x}) \approx p_{\text{data}}(\mathbf{x})$ under some regularity conditions.

2.2 DENOISING DIFFUSION PROBABILISTIC MODELS (DDPM)

Sohl-Dickstein et al. (2015); Ho et al. (2020) consider a sequence of positive noise scales $0 < \beta_1, \beta_2, \dots, \beta_N < 1$. For each training data point $\mathbf{x}_0 \sim p_{\text{data}}(\mathbf{x})$, a discrete Markov chain $\{\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N\}$ is constructed such that $p(\mathbf{x}_i | \mathbf{x}_{i-1}) = \mathcal{N}(\mathbf{x}_i; \sqrt{1 - \beta_i} \mathbf{x}_{i-1}, \beta_i \mathbf{I})$, and therefore $p_{\alpha_i}(\mathbf{x}_i | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_i; \sqrt{\alpha_i} \mathbf{x}_0, (1 - \alpha_i) \mathbf{I})$, where $\alpha_i := \prod_{j=1}^i (1 - \beta_j)$. Similar to SMLD, we can denote the perturbed data distribution as $p_{\alpha_i}(\tilde{\mathbf{x}}) := \int p_{\text{data}}(\mathbf{x}) p_{\alpha_i}(\tilde{\mathbf{x}} | \mathbf{x}) d\mathbf{x}$. The noise scales are prescribed such that $\mathbf{x}_N$ is approximately distributed according to $\mathcal{N}(\mathbf{0}, \mathbf{I})$. A variational Markov chain in the reverse direction is parameterized with $p_{\theta}(\mathbf{x}_{i-1} | \mathbf{x}_i) = \mathcal{N}(\mathbf{x}_{i-1}; \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta}(\mathbf{x}_i, i)), \beta_i \mathbf{I})$, and trained with a re-weighted variant of the evidence lower bound (ELBO):

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N (1 - \alpha_i) \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{p_{\alpha_i}(\tilde{\mathbf{x}} | \mathbf{x})} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}, i) - \nabla_{\tilde{\mathbf{x}}} \log p_{\alpha_i}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2]. \quad (3)$$

After solving Eq. (3) to get the optimal model $\mathbf{s}_{\theta^*}(\mathbf{x}, i)$, samples can be generated by starting from $\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and following the estimated reverse Markov chain as below

$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i, \quad i = N, N - 1, \dots, 1. \quad (4)$$

2.2 DENOISING DIFFUSION PROBABILISTIC MODELS (DDPM)

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N \sigma_i^2 \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{p_{\sigma_i}(\tilde{\mathbf{x}}|\mathbf{x})} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}, \sigma_i) - \nabla_{\tilde{\mathbf{x}}} \log p_{\sigma_i}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2]. \quad (1)$$

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N (1 - \alpha_i) \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{p_{\alpha_i}(\tilde{\mathbf{x}}|\mathbf{x})} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}, i) - \nabla_{\tilde{\mathbf{x}}} \log p_{\alpha_i}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2]. \quad (3)$$

$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i, \quad i = N, N-1, \dots, 1. \quad (4)$$

We call this method ancestral sampling, since it amounts to performing ancestral sampling from the graphical model $\prod_{i=1}^N p_{\theta}(\mathbf{x}_{i-1} | \mathbf{x}_i)$. The objective Eq. (3) described here is L_{simple} in Ho et al. (2020), written in a form to expose more similarity to Eq. (1). Like Eq. (1), Eq. (3) is also a weighted sum of denoising score matching objectives, which implies that the optimal model, $\mathbf{s}_{\theta^*}(\tilde{\mathbf{x}}, i)$, matches the score of the perturbed data distribution, $\nabla_{\tilde{\mathbf{x}}} \log p_{\alpha_i}(\tilde{\mathbf{x}} | \mathbf{x})$. Notably, the weights of the i -th summand in Eq. (1) and Eq. (3), namely σ_i^2 and $(1 - \alpha_i)$, are related to corresponding perturbation kernels in the same functional form: $\sigma_i^2 \propto 1/\mathbb{E}[\|\nabla_{\tilde{\mathbf{x}}} \log p_{\sigma_i}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2]$ and $(1 - \alpha_i) \propto 1/\mathbb{E}[\|\nabla_{\tilde{\mathbf{x}}} \log p_{\alpha_i}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2]$.

DDPM
$$p_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \boldsymbol{\Sigma}_{\theta}(\mathbf{x}_t, t)) \quad (1)$$

DDPM
$$\mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right] \quad (12)$$

Let's start here

SMLD - Langevin Dynamics sampling math 12/14

Sampling via the Langevin dynamics

$$dX(t) = -\nabla U(X(t))dt + \sigma dB_t \quad \text{Langevin equation}$$

Particles have the stationary distribution:

$$p_G(x) = \frac{1}{Z} \exp\left(-\frac{2U(x)}{\sigma^2}\right), \quad Z = \int dx \exp\left(-\frac{2U(x)}{\sigma^2}\right) \quad \text{Gibbs distribution}$$

We want to sample from $p(x) = \frac{\hat{p}(x)}{Z'}$

$$U(x) = -\log p(x), \quad \sigma = \sqrt{2}$$

$$p_G(x) = \frac{1}{Z} \exp\left(-\frac{-2\log p(x)}{2}\right) = p(x), \quad Z = \int dx \exp(\log p(x)) = 1$$

Note! $\nabla U(x) = -\nabla \log p(x) = -\nabla \log \hat{p}(x) - \nabla \log Z'$

The gradient of $U(x)$ (energy), does not depend on Z' .

The Langevin function give us some complicated dynamics for us to simulate

Sampling via the Langevin dynamics

Stochastic Differential Equation for sampling:

$$dX(t) = \nabla \log p(X(t))dt + \sqrt{2}dB_t$$

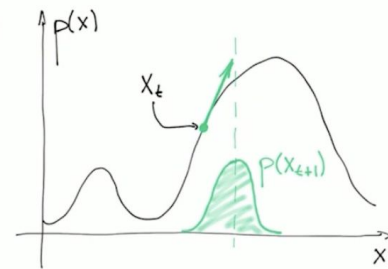
Discrete approximation:

$$X_{t+1} = X_t + dt \nabla \log p(X_t) + \mathcal{N}(0, 2dt)$$

More popular way:

$$X_{t+1} = X_t + \frac{\varepsilon}{2} \nabla \log p(X_t) + \mathcal{N}(0, \varepsilon)$$

$$X_t \sim p(x), \quad \forall t > t_\infty$$

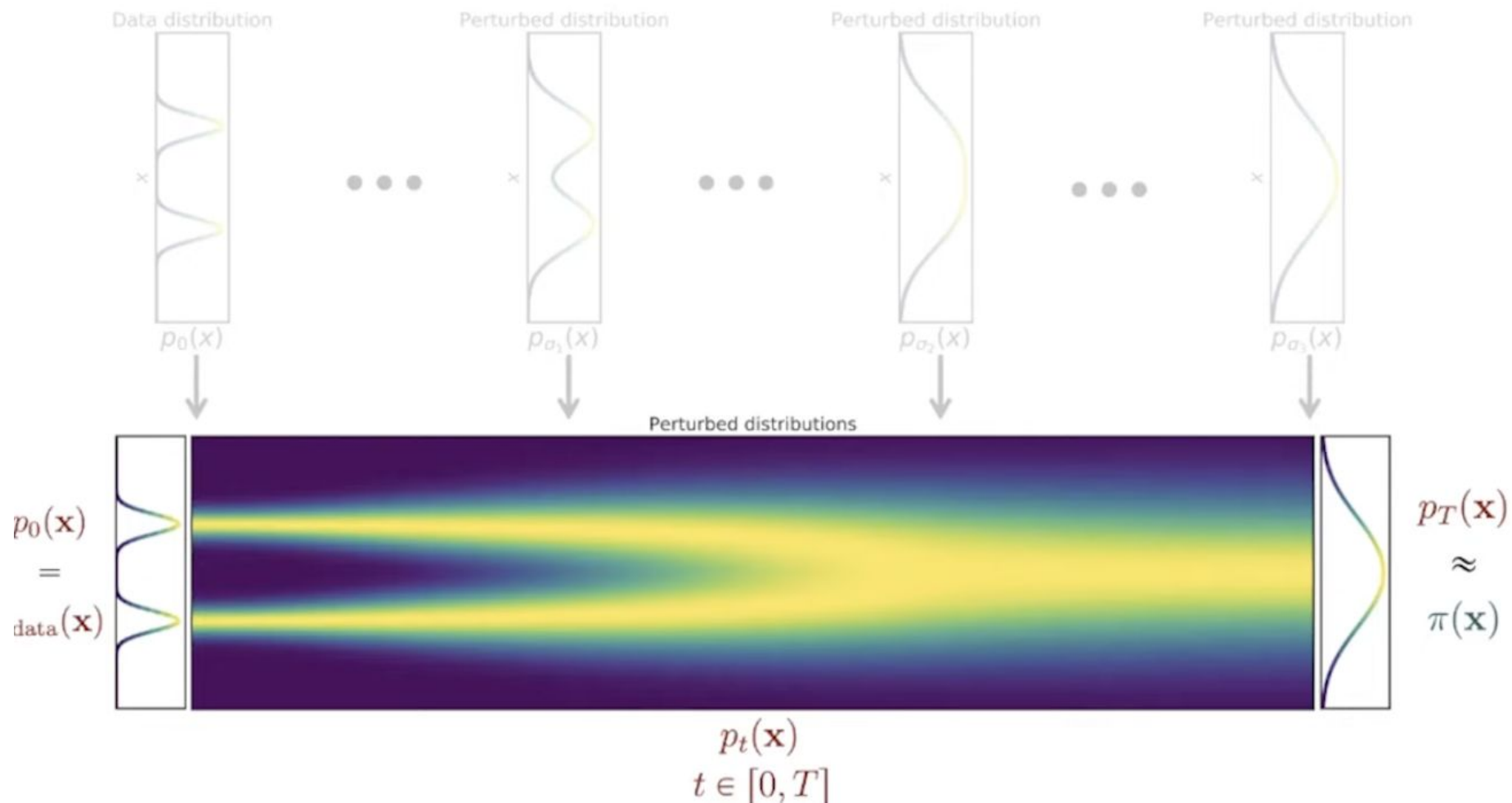


The gradient of $U(x)$ (energy), does not depend on Z' .

X moves to higher region of density

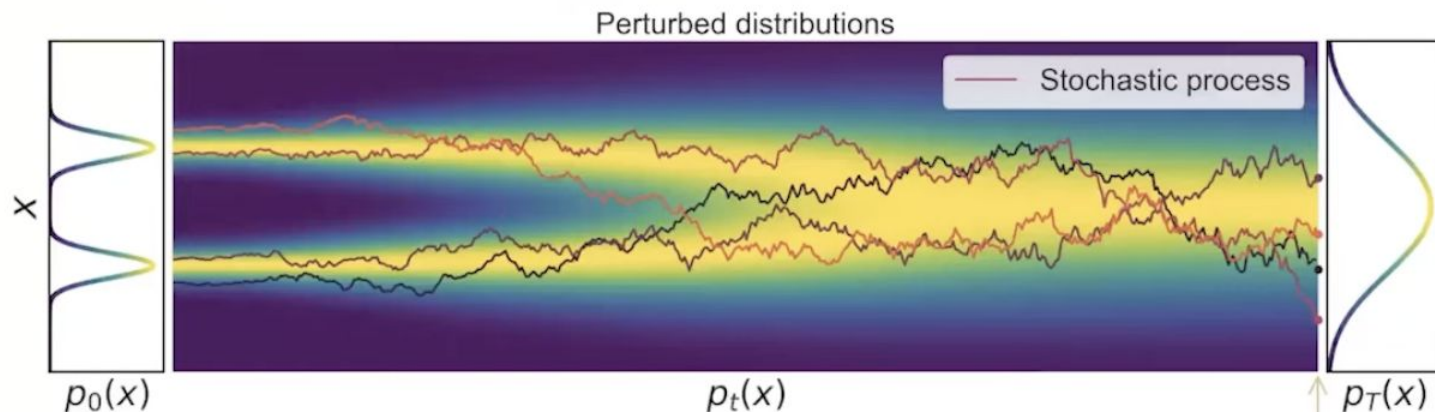
$$\mathbf{x}_{i+1} \leftarrow \mathbf{x}_i + \epsilon \nabla_{\mathbf{x}} \log p(\mathbf{x}) + \sqrt{2\epsilon} \mathbf{z}_i, \quad i = 0, 1, \dots, K, \quad (6)$$

Infinite noise levels



3.1 PERTURBING DATA WITH SDES

Perturbing data with stochastic processes



Stochastic process

$$\{\mathbf{x}_t\}_{t \in [0, T]}$$



Probability densities

$$\{p_t(\mathbf{x})\}_{t \in [0, T]}$$

Stochastic differential equation (SDE)

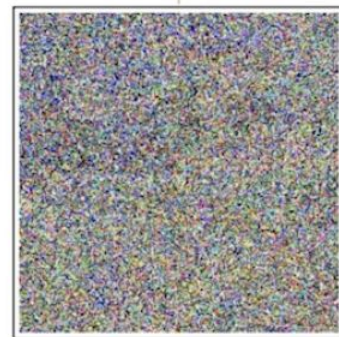
$$d\mathbf{x}_t = \boxed{f(\mathbf{x}_t, t)}dt + g(t)\boxed{d\mathbf{w}_t}$$

Deterministic drift

Infinitesimal noise

WLOG: Toy SDE

$$d\mathbf{x}_t = \sigma(t) d\mathbf{w}_t$$



$$p_T(\mathbf{x}) \approx \pi(\mathbf{x})$$

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}, \quad (8)$$

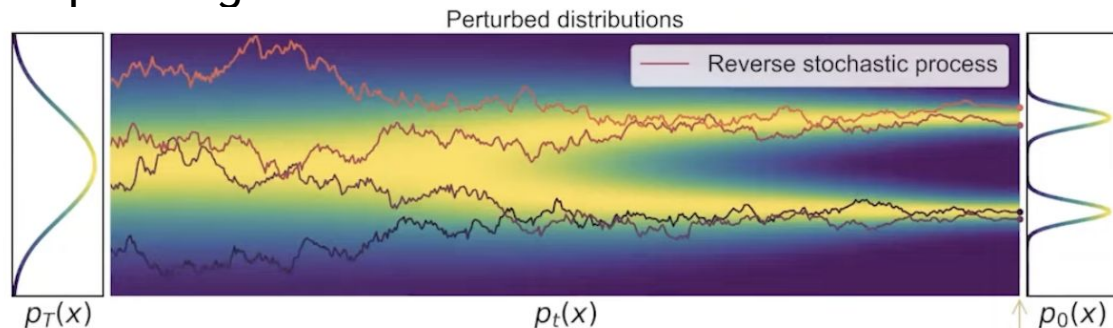
The SDE in (8) is hand designed, similarly to how we hand-designed $\sigma_1 < \sigma_2 < \dots < \sigma_L$ in the case of finite noise scales. There are numerous ways to add noise perturbations, and the choice of SDEs is not unique. For example, the following SDE

$$d\mathbf{x} = e^t d\mathbf{w} \quad (9)$$

perturbs data with a Gaussian noise of mean zero and exponentially growing variance, which is analogous to perturbing data with $\mathcal{N}(0, \sigma_1^2 I), \mathcal{N}(0, \sigma_2^2 I), \dots, \mathcal{N}(0, \sigma_L^2 I)$ when $\sigma_1 < \sigma_2 < \dots < \sigma_L$ is a geometric progression. Therefore, the SDE should be viewed as part of the model, much like $\{\sigma_1, \sigma_2, \dots, \sigma_L\}$. In [21], we provide three SDEs that generally work well for images: the Variance Exploding SDE (VE SDE), the Variance Preserving SDE (VP SDE), and the sub-VP SDE.

3.2 GENERATING SAMPLES BY REVERSING THE SDE

any SDE has a corresponding reverse SDE



Forward SDE (t: 0→T)

$$d\mathbf{x}_t = \sigma(t) d\mathbf{w}_t$$

Reverse SDE (t: T→0)

$$d\mathbf{x}_t = -\sigma(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t) dt + \sigma(t) \boxed{d\bar{\mathbf{w}}_t}$$

Infinitesimal noise in
the reverse time
direction

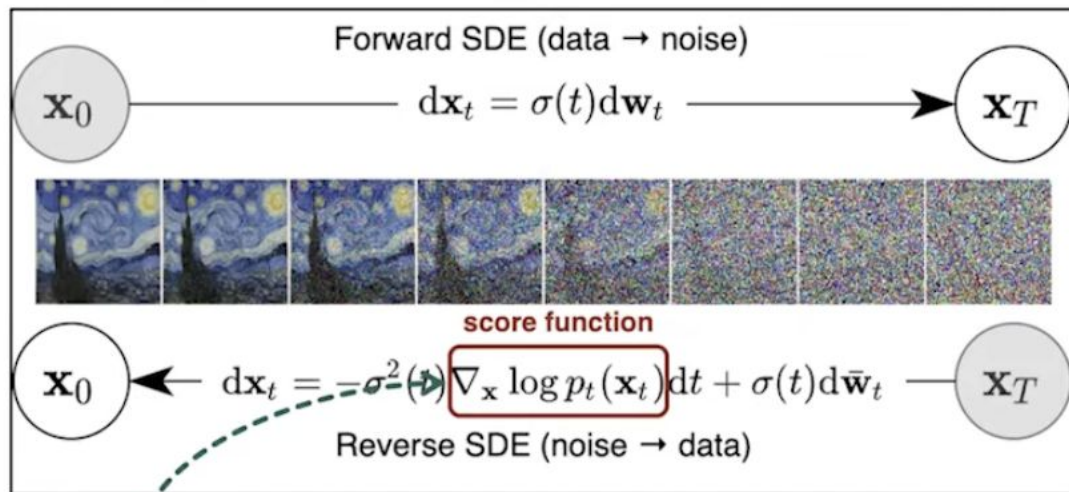


By starting from samples of $\mathbf{x}(T) \sim p_T$ and reversing the process, we can obtain samples $\mathbf{x}(0) \sim p_0$. A remarkable result from [Anderson \(1982\)](#) states that the reverse of a diffusion process is also a diffusion process, running backwards in time and given by the reverse-time SDE:

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x})]dt + g(t)d\bar{\mathbf{w}}, \quad (6)$$

where $\bar{\mathbf{w}}$ is a standard Wiener process when time flows backwards from T to 0, and dt is an infinitesimal negative timestep. Once the score of each marginal distribution, $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, is known for all t , we can derive the reverse diffusion process from Eq. (6) and simulate it to sample from p_0 .

Score-based generative modeling via SDEs



Time conditional
score model

$$s_{\theta}(\mathbf{x}, t)$$

$$\approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$$

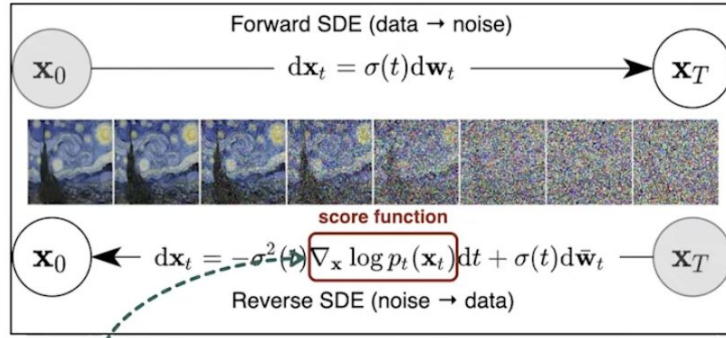
Training objective:

$$\mathbb{E}_{t \sim \text{Uniform}[0, T]} \left[\lambda(t) \mathbb{E}_{p_t(\mathbf{x})} \left[\left\| \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) - s_{\theta}(\mathbf{x}, t) \right\|_2^2 \right] \right]$$

Positive weighting
function

Score matching loss

3.3 ESTIMATING SCORES FOR THE SDE



Time conditional
score model

Training objective:

$$\mathbb{E}_{t \sim \text{Uniform}[0, T]} \left[\lambda(t) \mathbb{E}_{p_t(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}, t)\|_2^2] \right]$$

Positive weighting function
Score matching loss

$$\approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$$

score matching (Hyvärinen, 2005; Song et al., 2019a). To estimate $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, we can train a time-dependent score-based model $\mathbf{s}_{\theta}(\mathbf{x}, t)$ via a continuous generalization to Eqs. (1) and (3):

$$\theta^* = \arg \min_{\theta} \mathbb{E}_t \left\{ \lambda(t) \mathbb{E}_{\mathbf{x}(0)} \mathbb{E}_{\mathbf{x}(t)|\mathbf{x}(0)} [\|\mathbf{s}_{\theta}(\mathbf{x}(t), t) - \nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) | \mathbf{x}(0))\|_2^2] \right\}. \quad (7)$$

Here $\lambda : [0, T] \rightarrow \mathbb{R}_{>0}$ is a positive weighting function, t is uniformly sampled over $[0, T]$, $\mathbf{x}(0) \sim p_0(\mathbf{x})$ and $\mathbf{x}(t) \sim p_{0t}(\mathbf{x}(t) | \mathbf{x}(0))$. With sufficient data and model capacity, score matching ensures that the optimal solution to Eq. (7), denoted by $\mathbf{s}_{\theta^*}(\mathbf{x}, t)$, equals $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ for almost all $\mathbf{x}$ and t . As in SMLD and DDPM, we can typically choose $\lambda \propto 1/\mathbb{E}[\|\nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) | \mathbf{x}(0))\|_2^2]$. Note that Eq. (7) uses denoising score matching, but other score matching objectives, such as sliced score matching (Song et al., 2019a) and finite-difference score matching (Pang et al., 2020) are also applicable here.

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N \sigma_i^2 \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{p_{\sigma_i}(\tilde{\mathbf{x}}|\mathbf{x})} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}, \sigma_i) - \nabla_{\tilde{\mathbf{x}}} \log p_{\sigma_i}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2]. \quad (1)$$

$$\theta^* = \arg \min_{\theta} \sum_{i=1}^N (1 - \alpha_i) \mathbb{E}_{p_{\text{data}}(\mathbf{x})} \mathbb{E}_{p_{\alpha_i}(\tilde{\mathbf{x}}|\mathbf{x})} [\|\mathbf{s}_{\theta}(\tilde{\mathbf{x}}, i) - \nabla_{\tilde{\mathbf{x}}} \log p_{\alpha_i}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2]. \quad (3)$$

- Time-dependent score-based model

$$\mathbf{s}_{\theta}(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$$

- Training:

$$\mathbb{E}_{t \in \mathcal{U}(0, T)} [\lambda(t) \mathbb{E}_{p_t(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}, t)\|_2^2]]$$

- Reverse-time SDE

$$d\mathbf{x} = -\sigma^2(t) \mathbf{s}_{\theta}(\mathbf{x}, t) dt + \sigma(t) d\bar{\mathbf{w}}$$

- Euler-Maruyama (analogous to Euler for ODEs)

$$\mathbf{x} \leftarrow \mathbf{x} - \sigma(t)^2 \mathbf{s}_{\theta}(\mathbf{x}, t) \Delta t + \sigma(t) \mathbf{z}$$

$$(\mathbf{z} \sim \mathcal{N}(\mathbf{0}, |\Delta t| \mathbf{I}))$$

$$t \leftarrow t + \Delta t$$

3 SCORE-BASED GENERATIVE MODELING WITH SDE

Generalize this idea further to an infinite number of noise scales, such that perturbed data distributions evolve according to an SDE as the noise intensifies.

This diffusion process can be modeled as the solution to an Itô SDE:

$\{\mathbf{x}(t)\}_{t=0}^T$ indexed by a continuous time variable $t \in [0, T]$ $d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w},$ (5)

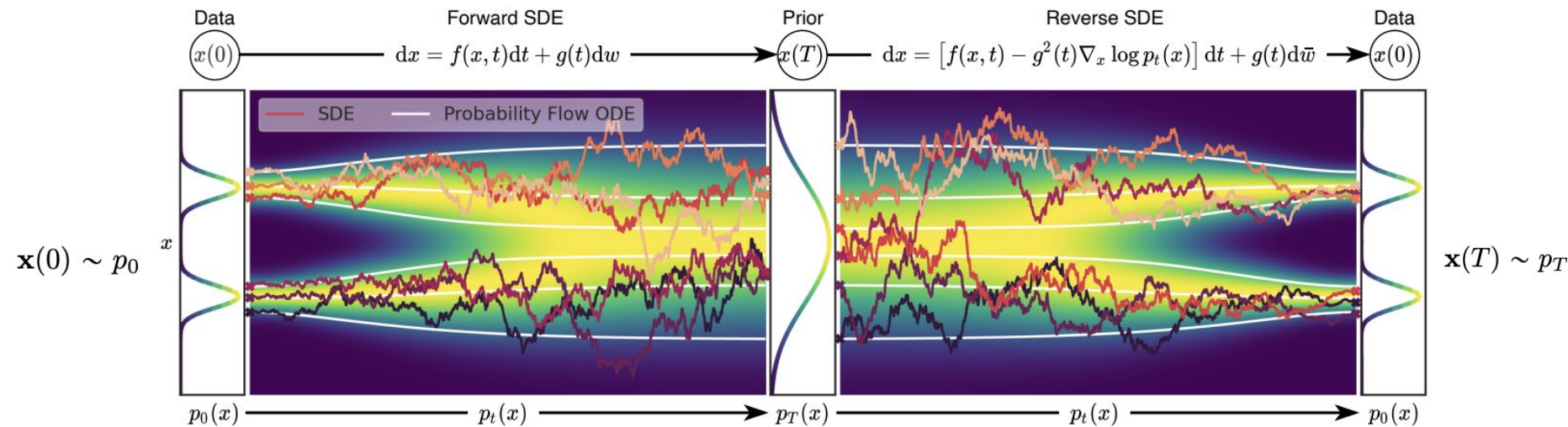


Figure 2: **Overview of score-based generative modeling through SDEs.** We can map data to a noise distribution (the prior) with an SDE (Section 3.1), and reverse this SDE for generative modeling (Section 3.2). We can also reverse the associated probability flow ODE (Section 4.3), which yields a deterministic process that samples from the same distribution as the SDE. Both the reverse-time SDE and probability flow ODE can be obtained by estimating the score $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ (Section 3.3).

3.1 More property

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}, \quad (5)$$

the *diffusion* coefficient of $\mathbf{x}(t)$. For ease of presentation we assume the diffusion coefficient is a scalar (instead of a $d \times d$ matrix) and does not depend on $\mathbf{x}$, but our theory can be generalized to hold in those cases (see Appendix A). The SDE has a unique strong solution as long as the coefficients are globally Lipschitz in both state and time (Øksendal, 2003). We hereafter denote by $p_t(\mathbf{x})$ the probability density of $\mathbf{x}(t)$, and use $p_{st}(\mathbf{x}(t) \mid \mathbf{x}(s))$ to denote the transition kernel from $\mathbf{x}(s)$ to $\mathbf{x}(t)$, where $0 \leq s < t \leq T$.

Typically, p_T is an unstructured prior distribution that contains no information of p_0 , such as a Gaussian distribution with fixed mean and variance. There are various ways of designing the SDE in Eq. (5) such that it diffuses the data distribution into a fixed prior distribution. We provide several examples later in Section 3.4 that are derived from continuous generalizations of SMLD and DDPM.

In addition, there are two special properties of our reverse SDE that allow for even more flexible sampling methods:

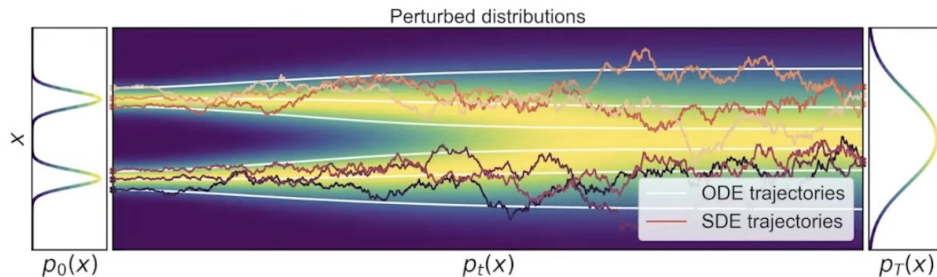
- We have an estimate of $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$ via our time-dependent score-based model $\mathbf{s}_\theta(\mathbf{x}, t)$.
- We only care about sampling from each marginal distribution $p_t(\mathbf{x})$. Samples obtained at different time steps can have arbitrary correlations and do not have to form a particular trajectory sampled from the reverse SDE.

As a consequence of these two properties, we can apply MCMC approaches to fine-tune the trajectories obtained from numerical SDE solvers. Specifically, we propose **Predictor-Corrector samplers**. The **predictor** can be any numerical SDE solver that predicts $\mathbf{x}(t + \Delta t) \sim p_{t+\Delta t}(\mathbf{x})$ from an existing sample $\mathbf{x}(t) \sim p_t(\mathbf{x})$. The **corrector** can be any MCMC procedure that solely relies on the score function, such as Langevin dynamics and Hamiltonian Monte Carlo.

At each step of the Predictor-Corrector sampler, we first use the predictor to choose a proper step size $\Delta t < 0$, and then predict $\mathbf{x}(t + \Delta t)$ based on the current sample $\mathbf{x}(t)$. Next, we run several corrector steps to improve the sample $\mathbf{x}(t + \Delta t)$ according to our score-based model $\mathbf{s}_\theta(\mathbf{x}, t + \Delta t)$, so that $\mathbf{x}(t + \Delta t)$ becomes a higher-quality sample from $p_{t+\Delta t}(\mathbf{x})$.

In [21], we show it is possible to convert any SDE into an ordinary differential equation (ODE) without changing its marginal distributions $\{p_t(\mathbf{x})\}_{t \in [0, T]}$. Thus by solving this ODE, we can sample from the same distributions as the reverse SDE. The corresponding ODE of an SDE is named probability flow ODE [21], given by

Converting the SDE to an ODE



The trajectory of SDE and ODE looks different from each other, but actually they can share the same set of marginal densities.

In other words, trajectories obtained by solving the probability flow ODE have the same marginal distributions as the SDE trajectories.

When the score function is approximated by the time-dependent score-based model, which is typically a neural network, this is an example of a neural ODE (Chen et al., 2018).

SDE **Ordinary differential equation (probability flow ODE)**

$$d\mathbf{x}_t = \sigma(t) d\mathbf{w}_t \longleftrightarrow \frac{d\mathbf{x}_t}{dt} = -\frac{1}{2}\sigma(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)$$

$\downarrow$
Score function
 $\approx s_{\theta}(\mathbf{x}, t)$

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + \mathbf{G}(\mathbf{x}, t)d\mathbf{w}, \quad (15)$$

According to (Anderson, 1982), the reverse-time SDE is given by (cf., Eq. (6))

$$d\mathbf{x} = \{\mathbf{f}(\mathbf{x}, t) - \nabla \cdot [\mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^T] - \mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^T \nabla_{\mathbf{x}} \log p_t(\mathbf{x})\}dt + \mathbf{G}(\mathbf{x}, t)d\bar{\mathbf{w}}, \quad (16)$$

The probability flow ODE corresponding to Eq. (15) has the following form (cf., Eq. (13), see a detailed derivation in Appendix D.1):

$$d\mathbf{x} = \left\{ \mathbf{f}(\mathbf{x}, t) - \frac{1}{2} \nabla \cdot [\mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^T] - \frac{1}{2} \mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^T \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right\} dt. \quad (17)$$

D.1 DERIVATION

The idea of probability flow ODE is inspired by Maoutsa et al. (2020), and one can find the derivation of a simplified case therein. Below we provide a derivation for the fully general ODE in Eq. (17). We consider the SDE in Eq. (15), which possesses the following form:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + \mathbf{G}(\mathbf{x}, t)d\mathbf{w},$$

where $\mathbf{f}(\cdot, t) : \mathbb{R}^d \rightarrow \mathbb{R}^d$ and $\mathbf{G}(\cdot, t) : \mathbb{R}^d \rightarrow \mathbb{R}^{d \times d}$. The marginal probability density $p_t(\mathbf{x}(t))$ evolves according to Kolmogorov's forward equation (Fokker-Planck equation) (Øksendal, 2003)

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = - \sum_{i=1}^d \frac{\partial}{\partial x_i} [f_i(\mathbf{x}, t)p_t(\mathbf{x})] + \frac{1}{2} \sum_{i=1}^d \sum_{j=1}^d \frac{\partial^2}{\partial x_i \partial x_j} \left[\sum_{k=1}^d G_{ik}(\mathbf{x}, t)G_{jk}(\mathbf{x}, t)p_t(\mathbf{x}) \right]. \quad (35)$$

We can easily rewrite Eq. (35) to obtain

$$\begin{aligned} \frac{\partial p_t(\mathbf{x})}{\partial t} &= - \sum_{i=1}^d \frac{\partial}{\partial x_i} [f_i(\mathbf{x}, t)p_t(\mathbf{x})] + \frac{1}{2} \sum_{i=1}^d \sum_{j=1}^d \frac{\partial^2}{\partial x_i \partial x_j} \left[\sum_{k=1}^d G_{ik}(\mathbf{x}, t)G_{jk}(\mathbf{x}, t)p_t(\mathbf{x}) \right] \\ &= - \sum_{i=1}^d \frac{\partial}{\partial x_i} [f_i(\mathbf{x}, t)p_t(\mathbf{x})] + \frac{1}{2} \sum_{i=1}^d \frac{\partial}{\partial x_i} \left[\sum_{j=1}^d \frac{\partial}{\partial x_j} \left[\sum_{k=1}^d G_{ik}(\mathbf{x}, t)G_{jk}(\mathbf{x}, t)p_t(\mathbf{x}) \right] \right]. \end{aligned} \quad (36)$$

Note that

$$\begin{aligned} &\sum_{j=1}^d \frac{\partial}{\partial x_j} \left[\sum_{k=1}^d G_{ik}(\mathbf{x}, t)G_{jk}(\mathbf{x}, t)p_t(\mathbf{x}) \right] \\ &= \sum_{j=1}^d \frac{\partial}{\partial x_j} \left[\sum_{k=1}^d G_{ik}(\mathbf{x}, t)G_{jk}(\mathbf{x}, t) \right] p_t(\mathbf{x}) + \sum_{j=1}^d \sum_{k=1}^d G_{ik}(\mathbf{x}, t)G_{jk}(\mathbf{x}, t)p_t(\mathbf{x}) \frac{\partial}{\partial x_j} \log p_t(\mathbf{x}) \\ &= p_t(\mathbf{x}) \nabla \cdot [\mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top] + p_t(\mathbf{x}) \mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top \nabla_{\mathbf{x}} \log p_t(\mathbf{x}), \end{aligned}$$

The probability flow ODE corresponding to Eq. (15) has the following form (cf., Eq. (13), see a detailed derivation in Appendix D.1):

$$d\mathbf{x} = \left\{ \mathbf{f}(\mathbf{x}, t) - \frac{1}{2} \nabla \cdot [\mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top] - \frac{1}{2} \mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right\} dt. \quad (17)$$

based on which we can continue the rewriting of Eq. (36) to obtain

$$\begin{aligned} \frac{\partial p_t(\mathbf{x})}{\partial t} &= - \sum_{i=1}^d \frac{\partial}{\partial x_i} [f_i(\mathbf{x}, t)p_t(\mathbf{x})] + \frac{1}{2} \sum_{i=1}^d \frac{\partial}{\partial x_i} \left[\sum_{j=1}^d \frac{\partial}{\partial x_j} \left[\sum_{k=1}^d G_{ik}(\mathbf{x}, t)G_{jk}(\mathbf{x}, t)p_t(\mathbf{x}) \right] \right] \\ &= - \sum_{i=1}^d \frac{\partial}{\partial x_i} [f_i(\mathbf{x}, t)p_t(\mathbf{x})] \\ &\quad + \frac{1}{2} \sum_{i=1}^d \frac{\partial}{\partial x_i} \left[p_t(\mathbf{x}) \nabla \cdot [\mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top] + p_t(\mathbf{x}) \mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] \\ &= - \sum_{i=1}^d \frac{\partial}{\partial x_i} \left\{ f_i(\mathbf{x}, t)p_t(\mathbf{x}) \right. \\ &\quad \left. - \frac{1}{2} \left[\nabla \cdot [\mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top] + \mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] p_t(\mathbf{x}) \right\} \\ &= - \sum_{i=1}^d \frac{\partial}{\partial x_i} [\tilde{f}_i(\mathbf{x}, t)p_t(\mathbf{x})], \end{aligned} \quad (37)$$

where we define

$$\tilde{\mathbf{f}}(\mathbf{x}, t) := \mathbf{f}(\mathbf{x}, t) - \frac{1}{2} \nabla \cdot [\mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top] - \frac{1}{2} \mathbf{G}(\mathbf{x}, t)\mathbf{G}(\mathbf{x}, t)^\top \nabla_{\mathbf{x}} \log p_t(\mathbf{x}).$$

Inspecting Eq. (37), we observe that it equals Kolmogorov's forward equation of the following SDE with $\tilde{\mathbf{G}}(\mathbf{x}, t) := \mathbf{0}$ (Kolmogorov's forward equation in this case is also known as the Liouville equation.)

$$d\mathbf{x} = \tilde{\mathbf{f}}(\mathbf{x}, t)dt + \tilde{\mathbf{G}}(\mathbf{x}, t)d\mathbf{w},$$

which is essentially an ODE:

$$d\mathbf{x} = \tilde{\mathbf{f}}(\mathbf{x}, t)dt,$$

same as the probability flow ODE given by Eq. (17). Therefore, we have shown that the probability flow ODE Eq. (17) induces the same marginal probability density $p_t(\mathbf{x})$ as the SDE in Eq. (15).

Computing the exact likelihood with ODEs

Likelihood:

$$\pi(\mathbf{x}) \xrightarrow[t : T \rightarrow 0]{s_{\theta}(\mathbf{x}, t)} p_{\theta}(\mathbf{x})$$

Applications of likelihood:

- Lossless compression (Witten et al. 1987, Townsend et al. 2019)
- Unsupervised anomaly detection (Pimental et al. 2014, **Song** et al. 2018)
- Generative classification (Ng & Jordan 2002, Zimmermann, Schott, **Song**, et al. 2021)
- Density estimation (Silverman, 1986)

Probability flow ODE allows exact likelihood computation:

$$\log p_{\theta}(\mathbf{x}_0) = \log \pi(\mathbf{x}_T) - \underbrace{\frac{1}{2} \int_0^T}_{\text{ODE solver}} \underbrace{\sigma(t)^2 \text{trace}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}, t))}_{\text{Unbiased estimator}} dt$$

(Change-of-variable formula for ODEs, Chen et al. 2018)

[**Song** et al. ICLR 2021 (Outstanding Paper Award)]

Solving ODEs for Sampling

Model	NLL Test ↓	FID ↓
RealNVP (Dinh et al., 2016)	3.49	-
iResNet (Behrmann et al., 2019)	3.45	-
Glow (Kingma & Dhariwal, 2018)	3.35	-
MintNet (Song et al., 2019b)	3.32	-
Residual Flow (Chen et al., 2019)	3.28	46.37
FFJORD (Grathwohl et al., 2018)	3.40	-
Flow++ (Ho et al., 2019)	3.29	-
DDPM (L) (Ho et al., 2020)	$\leq 3.70^*$	13.51
DDPM (L_{simple}) (Ho et al., 2020)	$\leq 3.75^*$	3.17
DDPM	3.28	3.37
DDPM cont. (VP)	3.21	3.69
DDPM cont. (sub-VP)	3.05	3.56
DDPM++ cont. (VP)	3.16	3.93
DDPM++ cont. (sub-VP)	3.02	3.16
DDPM++ cont. (deep, VP)	3.13	3.08
DDPM++ cont. (deep, sub-VP)	2.99	2.92

models trained
with score matching

black-box ODE
Solvers for sampling

Efficient maximum likelihood training

Theorem (informal): Connection between the Kullback-Leibler (KL) divergence and score matching. Different weighting function (likelihood weighting)

“Likelihood weighting”

$$\text{KL}(p_{\text{data}} \parallel p_{\theta}) \leq \frac{1}{2} \mathbb{E}_{t \sim \text{Uniform}[0, T]} [\overset{\uparrow}{\sigma(t)^2}] \mathbb{E}_{p_t(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) - \mathbf{s}_{\theta}(\mathbf{x}, t)\|_2^2]$$
$$+ \text{KL}(p_T \parallel \pi) \approx 0$$

By minimizing the score matching loss function, with this particular weighting function, we're actually implicitly maximizing likelihood.



Efficient surrogate loss for maximum likelihood training

3.3 VE SDE

The noise perturbations used in SMLD and DDPM can be regarded as discretizations of two different SDEs. Below we provide a brief discussion and relegate more details to Appendix B.

When using a total of N noise scales, each perturbation kernel $p_{\sigma_i}(\mathbf{x} \mid \mathbf{x}_0)$ of SMLD corresponds to the distribution of $\mathbf{x}_i$ in the following Markov chain:

$$\mathbf{x}_i = \mathbf{x}_{i-1} + \sqrt{\sigma_i^2 - \sigma_{i-1}^2} \mathbf{z}_{i-1}, \quad i = 1, \dots, N, \quad (8)$$

where $\mathbf{z}_{i-1} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, and we have introduced $\sigma_0 = 0$ to simplify the notation. In the limit of $N \rightarrow \infty$, $\{\sigma_i\}_{i=1}^N$ becomes a function $\sigma(t)$, $\mathbf{z}_i$ becomes $\mathbf{z}(t)$, and the Markov chain $\{\mathbf{x}_i\}_{i=1}^N$ becomes a continuous stochastic process $\{\mathbf{x}(t)\}_{t=0}^1$, where we have used a continuous time variable $t \in [0, 1]$ for indexing, rather than an integer i . The process $\{\mathbf{x}(t)\}_{t=0}^1$ is given by the following SDE

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w}. \quad (9)$$

3.3 VP SDE

Likewise for the perturbation kernels $\{p_{\alpha_i}(\mathbf{x} \mid \mathbf{x}_0)\}_{i=1}^N$ of DDPM, the discrete Markov chain is

$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}, \quad i = 1, \dots, N. \quad (10)$$

As $N \rightarrow \infty$, Eq. (10) converges to the following SDE,

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}. \quad (11)$$

Therefore, the noise perturbations used in SMLD and DDPM correspond to discretizations of SDEs Eqs. (9) and (11). Interestingly, the SDE of Eq. (9) always gives a process with exploding variance when $t \rightarrow \infty$, whilst the SDE of Eq. (11) yields a process with a fixed variance of one when the initial distribution has unit variance (proof in Appendix B). Due to this difference, we hereafter refer to Eq. (9) as the Variance Exploding (VE) SDE, and Eq. (11) the Variance Preserving (VP) SDE.

3.3 sub-VP SDE

Inspired by the VP SDE, we propose a new type of SDEs which perform particularly well on likelihoods (see Section 4.3), given by

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)(1 - e^{-2\int_0^t \beta(s)ds})}d\mathbf{w}. \quad (12)$$

When using the same $\beta(t)$ and starting from the same initial distribution, the variance of the stochastic process induced by Eq. (12) is always bounded by the VP SDE at every intermediate time step (proof in Appendix B). For this reason, we name Eq. (12) the sub-VP SDE.

Since VE, VP and sub-VP SDEs all have affine drift coefficients, their perturbation kernels $p_{0t}(\mathbf{x}(t) | \mathbf{x}(0))$ are all Gaussian and can be computed in closed-forms, as discussed in Section 3.3. This makes training with Eq. (7) particularly efficient.

4 SOLVING THE REVERSE SDE

Table 1: Comparing different reverse-time SDE solvers on CIFAR-10. Shaded regions are obtained with the same computation (number of score function evaluations). Mean and standard deviation are reported over five sampling runs. “P1000” or “P2000”: predictor-only samplers using 1000 or 2000 steps. “C2000”: corrector-only samplers using 2000 steps. “PC1000”: Predictor-Corrector (PC) samplers using 1000 predictor and 1000 corrector steps.

		Variance Exploding SDE (SMLD)				Variance Preserving SDE (DDPM)			
FID↓	Sampler Predictor	P1000	P2000	C2000	PC1000	P1000	P2000	C2000	PC1000
		ancestral sampling	4.98 $\pm$.06	4.88 $\pm$.06	20.43 $\pm$.07	3.62 $\pm$.03	3.24 $\pm$.02	3.24 $\pm$.02	3.21 $\pm$.02
	reverse diffusion	4.79 $\pm$.07	4.74 $\pm$.08		3.60 $\pm$.02	3.21 $\pm$.02	3.19 $\pm$.02	19.06 $\pm$.06	3.18 $\pm$.01
	probability flow	15.41 $\pm$.15	10.54 $\pm$.08		3.51 $\pm$.04	3.59 $\pm$.04	3.23 $\pm$.03		3.06 $\pm$.03

4.1 GENERAL-PURPOSE NUMERICAL SDE SOLVERS

Numerical solvers provide approximate trajectories from SDEs. Many general-purpose numerical methods exist for solving SDEs, such as Euler-Maruyama and stochastic Runge-Kutta methods (Kloeden & Platen, 2013), which correspond to different discretizations of the stochastic dynamics. We can apply any of them to the reverse-time SDE for sample generation.

Ancestral sampling, the sampling method of DDPM (Eq. (4)), actually corresponds to one special discretization of the reverse-time VP SDE (Eq. (11)) (see Appendix E). Deriving the ancestral sampling rules for new SDEs, however, can be non-trivial. To remedy this, we propose *reverse diffusion samplers* (details in Appendix E), which discretize the reverse-time SDE in the same way as the forward one, and thus can be readily derived given the forward discretization. As shown in Table 1, reverse diffusion samplers perform slightly better than ancestral sampling for both SMLD and DDPM models on CIFAR-10 (DDPM-type ancestral sampling is also applicable to SMLD models, see Appendix F.)

4.2 PREDICTOR-CORRECTOR SAMPLERS

Unlike generic SDEs, we have additional information that can be used to improve solutions. Since we have a score-based model $\mathbf{s}_{\theta^*}(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, we can employ score-based MCMC approaches, such as Langevin MCMC (Parisi, 1981; Grenander & Miller, 1994) or HMC (Neal et al., 2011) to sample from p_t directly, and correct the solution of a numerical SDE solver.

Specifically, at each time step, the numerical SDE solver first gives an estimate of the sample at the next time step, playing the role of a “predictor”. Then, the score-based MCMC approach corrects the marginal distribution of the estimated sample, playing the role of a “corrector”. The idea is analogous to Predictor-Corrector methods, a family of numerical continuation techniques for solving systems of equations (Allgower & Georg, 2012), and we similarly name our hybrid sampling algorithms *Predictor-Corrector* (PC) samplers. Please find pseudo-code and a complete description in Appendix G. PC samplers generalize the original sampling methods of SMLD and DDPM: the former uses an identity function as the predictor and annealed Langevin dynamics as the corrector, while the latter uses ancestral sampling as the predictor and identity as the corrector.

4.3 PROBABILITY FLOW AND CONNECTION TO NEURAL ODES

Score-based models enable another numerical method for solving the reverse-time SDE. For all diffusion processes, there exists a corresponding *deterministic process* whose trajectories share the same marginal probability densities $\{p_t(\mathbf{x})\}_{t=0}^T$ as the SDE. This deterministic process satisfies an ODE (more details in Appendix D.1):

$$\underline{d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt,} \quad (13)$$

which can be determined from the SDE once scores are known. We name the ODE in Eq. (13) the *probability flow ODE*. When the score function is approximated by the time-dependent score-based model, which is typically a neural network, this is an example of a neural ODE (Chen et al., 2018).

Exact likelihood computation Leveraging the connection to neural ODEs, we can compute the density defined by Eq. (13) via the instantaneous change of variables formula (Chen et al., 2018). This allows us to compute the *exact likelihood on any input data* (details in Appendix D.2). As an example, we report negative log-likelihoods (NLLs) measured in bits/dim on the CIFAR-10 dataset in Table 2. We compute log-likelihoods on uniformly dequantized data, and only compare to models evaluated in the same way (omitting models evaluated with variational dequantization (Ho et al., 2019) or discrete data), except for DDPM (L/L_{simple}) whose ELBO values (annotated with *) are reported on discrete data. Main results: (i) For the same DDPM model in Ho et al. (2020), we obtain better bits/dim than ELBO, since our likelihoods are exact; (ii) Using the same architecture, we trained another DDPM model with the continuous objective in Eq. (7) (*i.e.*, DDPM cont.), which further improves the likelihood; (iii) With sub-VP SDEs, we always get higher likelihoods compared to VP SDEs; (iv) With improved architecture (*i.e.*, DDPM++ cont., details in Section 4.4) and the sub-VP SDE, we can set a new record bits/dim of 2.99 on uniformly dequantized CIFAR-10 even *without maximum likelihood training*.

Manipulating latent representations By integrating Eq. (13), we can encode any datapoint $\mathbf{x}(0)$ into a latent space $\mathbf{x}(T)$. Decoding can be achieved by integrating a corresponding ODE for the reverse-time SDE. As is done with other invertible models such as neural ODEs and normalizing flows (Dinh et al., 2016; Kingma & Dhariwal, 2018), we can manipulate this latent representation for image editing, such as interpolation, and temperature scaling (see Fig. 3 and Appendix D.4).

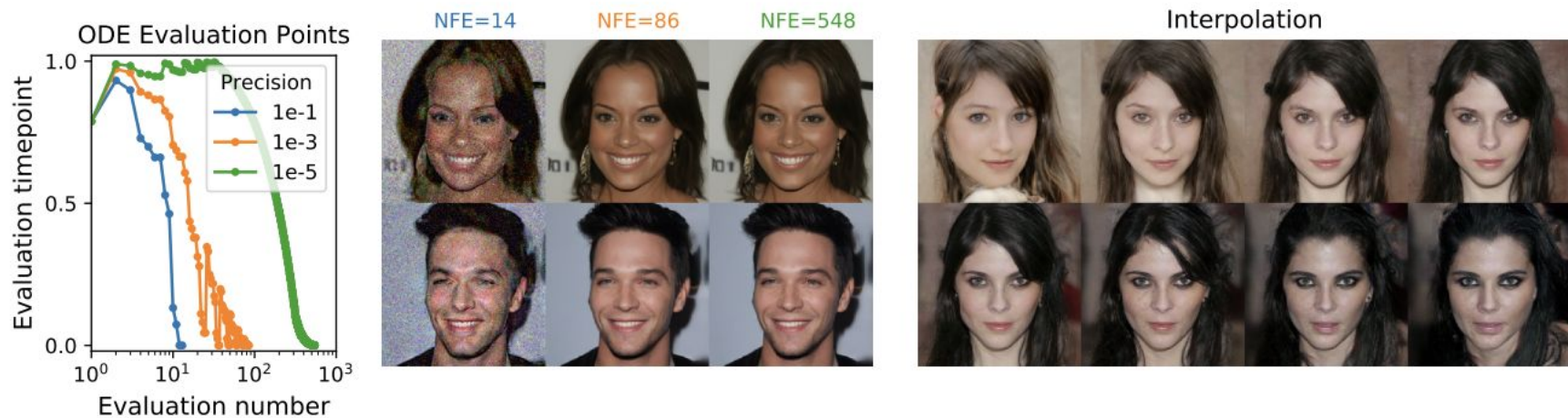


Figure 3: Probability flow ODE enables fast sampling with adaptive stepsizes as the numerical precision is varied (*left*), and reduces the number of score function evaluations (NFE) without harming quality (*middle*). The invertible mapping from latents to images allows for interpolations (*right*).

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}, \quad (5)$$

$$d\mathbf{x} = \left[\mathbf{f}(\mathbf{x}, t) - \frac{1}{2}g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt, \quad (13)$$

Uniquely identifiable encoding Unlike most current invertible models, our encoding is uniquely identifiable, meaning that with sufficient training data, model capacity, and optimization accuracy, the encoding for an input is uniquely determined by the data distribution (Roeder et al., 2020). This is because our forward SDE, Eq. (5), has no trainable parameters, and its associated probability flow ODE, Eq. (13), provides the same trajectories given perfectly estimated scores. We provide additional empirical verification on this property in Appendix D.5.

Efficient sampling As with neural ODEs, we can sample $\mathbf{x}(0) \sim p_0$ by solving Eq. (13) from different final conditions $\mathbf{x}(T) \sim p_T$. Using a fixed discretization strategy we can generate competitive samples, especially when used in conjunction with correctors (Table 1, “probability flow sampler”, details in Appendix D.3). Using a black-box ODE solver (Dormand & Prince, 1980) not only produces high quality samples (Table 2, details in Appendix D.4), but also allows us to explicitly trade-off accuracy for efficiency. With a larger error tolerance, the number of function evaluations can be reduced by over 90% without affecting the visual quality of samples (Fig. 3).

D.3 PROBABILITY FLOW SAMPLING

Suppose we have a forward SDE

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + \mathbf{G}(t)d\mathbf{w},$$

and one of its discretization

$$\mathbf{x}_{i+1} = \mathbf{x}_i + \mathbf{f}_i(\mathbf{x}_i) + \mathbf{G}_i \mathbf{z}_i, \quad i = 0, 1, \dots, N-1, \quad (41)$$

where $\mathbf{z}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. We assume the discretization schedule of time is fixed beforehand, and thus we absorb the dependency on Δt into the notations of $\mathbf{f}_i$ and $\mathbf{G}_i$. Using Eq. (17), we can obtain the following probability flow ODE:

$$d\mathbf{x} = \left\{ \mathbf{f}(\mathbf{x}, t) - \frac{1}{2} \mathbf{G}(t) \mathbf{G}(t)^\top \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right\} dt. \quad (42)$$

We may employ any numerical method to integrate the probability flow ODE backwards in time for sample generation. In particular, we propose a discretization in a similar functional form to Eq. (41):

$$\mathbf{x}_i = \mathbf{x}_{i+1} - \mathbf{f}_{i+1}(\mathbf{x}_{i+1}) + \frac{1}{2} \mathbf{G}_{i+1} \mathbf{G}_{i+1}^\top \mathbf{s}_{\theta^*}(\mathbf{x}_{i+1}, i+1), \quad i = 0, 1, \dots, N-1,$$

where the score-based model $\mathbf{s}_{\theta^*}(\mathbf{x}_i, i)$ is conditioned on the iteration number i . This is a deterministic iteration rule. Unlike reverse diffusion samplers or ancestral sampling, there is no additional randomness once the initial sample $\mathbf{x}_N$ is obtained from the prior distribution. When applied to SMLD models, we can get the following iteration rule for probability flow sampling:

$$\mathbf{x}_i = \mathbf{x}_{i+1} + \frac{1}{2} (\sigma_{i+1}^2 - \sigma_i^2) \mathbf{s}_{\theta^*}(\mathbf{x}_{i+1}, \sigma_{i+1}), \quad i = 0, 1, \dots, N-1. \quad (43)$$

Similarly, for DDPM models, we have

$$\mathbf{x}_i = (2 - \sqrt{1 - \beta_{i+1}}) \mathbf{x}_{i+1} + \frac{1}{2} \beta_{i+1} \mathbf{s}_{\theta^*}(\mathbf{x}_{i+1}, i+1), \quad i = 0, 1, \dots, N-1. \quad (44)$$

Table 2: NLLs and FIDs (ODE) on CIFAR-10.

Model	NLL Test ↓	FID ↓
RealNVP (Dinh et al., 2016)	3.49	-
iResNet (Behrmann et al., 2019)	3.45	-
Glow (Kingma & Dhariwal, 2018)	3.35	-
MintNet (Song et al., 2019b)	3.32	-
Residual Flow (Chen et al., 2019)	3.28	46.37
FFJORD (Grathwohl et al., 2018)	3.40	-
Flow++ (Ho et al., 2019)	3.29	-
DDPM (L) (Ho et al., 2020)	$\leq 3.70^*$	13.51
DDPM (L_{simple}) (Ho et al., 2020)	$\leq 3.75^*$	3.17
DDPM	3.28	3.37
DDPM cont. (VP)	3.21	3.69
DDPM cont. (sub-VP)	3.05	3.56
DDPM++ cont. (VP)	3.16	3.93
DDPM++ cont. (sub-VP)	3.02	3.16
DDPM++ cont. (deep, VP)	3.13	3.08
DDPM++ cont. (deep, sub-VP)	2.99	2.92

Table 3: CIFAR-10 sample quality.

Model	FID↓	IS↑
Conditional		
BigGAN (Brock et al., 2018)	14.73	9.22
StyleGAN2-ADA (Karras et al., 2020a)	2.42	10.14
Unconditional		
StyleGAN2-ADA (Karras et al., 2020a)	2.92	9.83
NCSN (Song & Ermon, 2019)	25.32	$8.87 \pm .12$
NCSNv2 (Song & Ermon, 2020)	10.87	$8.40 \pm .07$
DDPM (Ho et al., 2020)	3.17	$9.46 \pm .11$
DDPM++	2.78	9.64
DDPM++ cont. (VP)	2.55	9.58
DDPM++ cont. (sub-VP)	2.61	9.56
DDPM++ cont. (deep, VP)	2.41	9.68
DDPM++ cont. (deep, sub-VP)	2.41	9.57
NCSN++	2.45	9.73
NCSN++ cont. (VE)	2.38	9.83
NCSN++ cont. (deep, VE)	2.20	9.89